

University of
St Andrews

School of Computer Science
University of St Andrews

Multi-Agent AI System for Consulting Interview Simulation and Feedback

Dissertation submitted in partial fulfillment of the requirements for the degree of

MSc Artificial Intelligence

Student: Paloma Pérez de Madrid Laguna
Student ID: 250016561
Supervisor: Dr Ognjen Arandelovic
Submission date: August 2026

Abstract

This dissertation builds and compares two LLM-based systems that run and score consulting case interviews, a single-agent baseline that interviews, judges, and gives feedback in one call, and a multi-agent system that splits those roles across a dedicated Interviewer, Judge, and Feedback agent. Human-led case interview practice is scarce and expensive, so the project tests whether a specialised multi-agent architecture judges more reliably than a single model handling everything at once, and whether grounding the Judge in retrieval-augmented generation improves that reliability further. Both systems run on the same LangGraph framework and identical synthetic cases, and are evaluated together at agent, scenario, and experiment level through golden sets, repeated runs, and a manual human review. Neither system reaches meaningful agreement with human scoring, since both Judges collapse most ratings toward a single score, though the agentic system tracks evidence across turns far more consistently, at roughly double the computational cost, and grounding the Judge in retrieval improves accuracy on checkable dimensions like case-math and confidence calibration while also shifting its standard on clarity and concision. These results suggest multi-agent coordination pays off mainly when a task requires tracking state across a conversation rather than judging a single turn, and that retrieval-augmented grounding does not just add missing facts to a judge but can quietly recalibrate its standards on dimensions that have no external referent to check against.

Declaration

I hereby certify that this dissertation, which is approximately 15,000 words in length, has been composed by me, that it is the record of work carried out by me, and that it has not been submitted in any previous application for a degree. I confirm that AI tools were used to assist with editing and language refinement of the writing. At the code level, AI tools were used for grammar and language correction in code comments, as well as for minor refactoring of function and variable names to improve code readability. AI tools were not used to design or implement the system, nor to produce the analysis, results, or conclusions presented in this dissertation; these are my own work.

This project was conducted by me at the University of St Andrews from June 2026 to August 2026 towards fulfilment of the requirements of the University of St Andrews for the degree of MSc Artificial Intelligence under the supervision of Dr Ognjen Arandelovic.

In submitting this project report to the University of St Andrews, I give permission for it to be published online. I retain the copyright in this work.

Date: 11 of August of 2026

Signature: _____

Contents

Abstract **1**

Declaration **2**

1 Introduction **8**
1.1 Project Context 8
1.2 Research questions 9
1.3 Document Outline 9

2 Context & Related Work **11**
2.1 Consulting Case Interviews 11
2.2 Multi-agent systems 11
2.3 Prior work 12
2.4 LLMs as a judge 13
2.5 Retrieval-Augmented Generation 13
2.6 Research Gap 14

3 Proposed Solution **15**
3.1 Overview 15
3.1.1 Baseline and Agentic comparison 17
3.2 Architecture 18
3.2.1 Data 18
3.2.2 Model Selection 21
3.2.3 RAG System 22
3.2.4 Baseline System 23
3.2.5 Agentic System 25
3.3 Technical Implementation 26
3.3.1 Why LangGraph 27
3.3.2 States and Node Construction 27
3.4 System Constraints 29
3.4.1 Copyright 29
3.4.2 LLM limitations 31
3.4.3 Candidate as an LLM 32

4 Evaluation **34**
4.1 Agents . . . 34
4.1.1 Evaluation Dimensions . . . 34
4.1.2 Results . . . 36
4.1.3 Golden Sets . . . 38
4.2 Run . . . 39
4.2.1 Observations across the experiment . . . 39
4.3 Scenario . . . 41
4.3.1 Human review design . . . 42
4.3.2 Main patterns . . . 42
4.4 Experiment . . . 42
4.4.1 Scenario design . . . 43
4.4.2 Metrics . . . 44
4.4.3 MAE, Recall, Precision & QWK . . . 45
4.4.4 Confusion Matrix . . . 45
4.4.5 Accuracy by dimension . . . 47
4.5 RAG . . . 47
4.5.1 RAG Metrics . . . 50
4.5.2 RAG Ablation . . . 51
4.6 Limitation of the system . . . 53

5 Discussion **54**
5.1 When multi-agent coordination helps . . . 54
5.1.1 Task-Level Evidence on State Tracking and Single-Turn Judgement 54
5.1.2 The same boundary in the scenario review . . . 55
5.1.3 Scoring discrimination . . . 56
5.1.4 What the advantage costs . . . 56
5.2 The Judge calibration effect induced by RAG . . . 57
5.3 What this design cannot yet conclude . . . 58

6 Conclusion **60**
6.1 RQ1. Sustaining the interview and grounding the evaluation . . . 60
6.2 RQ2. Task conditions and computational cost . . . 60
6.3 RQ3. The effect of RAG grounding on the Judge . . . 61
6.4 Contributions . . . 62
6.5 Future work . . . 63

A Technical Implementation Details **65**
A.0.1 Prompt Engineering . . . 65
A.0.2 Engineering Challenges . . . 65

A.0.3 Persistence . . . 66
A.0.4 Workbench . . . 67
A.0.5 Code Base . . . 68

A Model Selection **70**

A Human Review of Run-by-Run Interview Quality **73**

A Proposed User Study **83**

A Testing **85**
A.1 Testing Scope . . . 85
A.2 Automated Tests . . . 85
A.2.1 Studio Logic . . . 85
A.2.2 Workbench and Persistence . . . 86
A.3 Integration and Smoke Tests . . . 86
A.4 Current Limitations . . . 86

A User Manual **87**
A.1 Requirements and Installation . . . 87
A.2 Configuring the LLM Roles . . . 87
A.3 Running Interview Batches . . . 88
A.4 The Workbench . . . 89
A.4.1 Runs and run detail . . . 89
A.4.2 Compare . . . 89
A.4.3 Human evaluation . . . 90
A.4.4 Experiment . . . 90
A.4.5 RAG Evaluation . . . 90
A.4.6 Agents . . . 90
A.5 Node-Level Evaluation Suites . . . 90
A.6 LangGraph Dev Server . . . 91
A.7 Automated Tests . . . 91

List of Figures

3.1 High-level view of the two-graph system. . . . 16
3.2 High-level view of the data . . . 19
3.3 Schema of a profile given as a scenario . . . 21

3.4 Baseline Graph Architecture 24
3.5 Agentic Graph Architecture 25
3.6 Common State (Fields both graphs read/write identically) 28
3.7 LangGraph States from Baseline Graph 29
3.8 LangGraph States from Agentic Graph 30

4.1 Overview of the evaluation pipeline. 35
4.2 Interviewer evaluation taxonomy: conversation situation mapped to the expected label, by category. 35
4.3 Judge evaluation taxonomy: `enough_evidence` broken down into quantitative metrics and the qualitative causes behind correct and incorrect verdicts. 36
4.4 Guardrail: agentic vs baseline accuracy across the four leak-resistance categories. 36
4.5 Evidence handling: agentic vs baseline accuracy across the five fact-management categories. 37
4.6 Socratic function: agentic vs baseline accuracy across the three question-type categories. 37
4.7 Turn control: agentic interviewer accuracy across the six wrap-up and readiness categories (no baseline equivalent). 38
4.8 Turn-by-turn trace of a single run (baseline, Vertex AI scenario) on the workbench's `run_trace` page. 40
4.9 Workbench scenario view, used to inspect repeated runs of the same scenario across both architectures. 41
4.10 Basic metrics shown on the workbench's `/experiment` 44
4.11 Accuracy by true reference score from workbench 45
4.12 MAE, Recall, Precision & QWK 46
4.13 Accuracy by dimension when the dashboard is filtered to reference scores 1–3 only. 48
4.14 Accuracy by dimension with all reference scores included. 49
4.15 Live retrieval quality dashboard. 50
4.16 Retrieval quality by query category, $K = 5$. 51

A.1 Workbench: experiment overview page. 68
A.2 Code base: module dependencies inside `main/studio/` and `main/web/`, and the on-disk data each side reads and writes. 69

A.1 Ethics Form 95

List of Tables

3.1 Structural differences between the baseline and agentic graphs. 17
3.2 Final model configuration by role. 22
3.3 Which nodes consult which RAG source, and when the excerpt they ask for becomes available. 23

4.1 Golden set sizes by node and category. 39
4.2 Framework used in the manual scenario-level review, with each dimension scored on a 1–4 scale. 42
4.3 Confusion matrix for the agentic graph. 46
4.4 Confusion matrix for the baseline graphs. 46
4.5 Score delta by dimension, `baseline` graph. Mean delta = without RAG − with RAG. 52
4.6 Score delta by dimension, `agentic` graph. Mean delta = without RAG − with RAG. 52

A.1 Workbench pages, grouped by task and by which store they read. . . . 67

A.1 Final model configuration by role. 71

A.1 Human evaluation framework for interviewer behaviour. Each dimension is rated from 1 (poor) to 4 (strong), adapted from Socratic dialogue principles proposed by Favero et al. [12]. 74

1 Introduction

1.1 Project Context

Ericsson, Krampe and Tesch-Römer [1] argue that expertise develops through deliberate practice, understood as repeated exposure to challenging tasks combined with immediate, personalized feedback on what to correct and how.Feedback of this kind is especially valuable in domains where judgment is exercised under uncertainty, as in consulting case interviews, even if its weight relative to other learning mechanisms varies across skills and contexts.

In a case interview, the candidate is placed in front of an open-ended business problem and asked to think out loud, structure the issue, decide what information to ask for, reason quantitatively, and land on a defensible recommendation. There is rarely a single correct answer; what is being evaluated is the quality of the reasoning process itself, how the candidate structures ambiguity, tests assumptions, and adapts when new facts are introduced [2].

This is where current preparation resources fall short. Two problems compound each other:

- **Human feedback is expensive and scarce.** The most reliable way to practice a case interview realistically is to work through it with someone else (a peer, a coach, or a professional) who can improvise a follow-up question, challenge an assumption, or judge whether an answer is actually well-reasoned.

- **Existing tools are static.** University casebooks and commercial prep guides are, by their own design, references. They synthesize frameworks, worked examples, and scoring rubrics into a fixed document. However the candidate studies them, the content itself does not respond to what the candidate says or does.

The consequence is a broken feedback loop: candidates have abundant material to study, but very little opportunity to practice under conditions that resemble the real interview and receive feedback.

Recent advances in large language models suggest a way to close this loop [3]. An LLM can hold a multi-turn conversation, adapt its questions to what the candidate has just said, and evaluate reasoning rather than surface form. This project investigates whether the same dynamic, conversational approach can be turned around and applied to training rather than evaluation. Instead of a static LLM judge scoring a fixed

transcript, an LLM-based system conducts the case interview itself in real time and produces feedback grounded in how the conversation actually unfolded.

To study this, I built two systems and compared them. The *baseline system* consists of a single agent that conducts the interview, evaluates the candidate's responses, and generates feedback, all in one role. The *agentic system* splits this into three agents: an *interviewer agent*, which presents the case and guides the conversation while encouraging structured reasoning; a *judge agent*, which continuously assesses whether the candidate's reasoning is sufficient and aligned with the case objectives, and signals to the interviewer which areas still need to be explored; and a *feedback agent*, which takes the full interaction together with the judge's evaluation and turns it into personalized, actionable feedback on the candidate's performance. The three agents interact without following a fixed sequence, until enough evidence has been gathered to assess the candidate's performance, at which point the feedback agent produces the final assessment of the case resolution.

1.2 Research questions

The main objective of this project is to design and evaluate an AI-based system that simulates a consulting case interview and closes the feedback loop described above by providing personalized, low-cost, on-demand practice.

This objective is broken down into the following research questions:

- RQ1. Can an LLM-based system sustain a multi-turn consulting case interview and produce an evaluation grounded in evidence from the transcript?
- RQ2. Under what task conditions does a fully specialised multi-agent architecture outperform a single-agent baseline on evaluation reliability, and at what computational cost?
- RQ3. What effect does grounding the system with retrieval-augmented generation (RAG) have on the Judge's evaluations?

1.3 Document Outline

Chapter 2 reviews the related work on LLM-based interviewing and evaluation and lays out the domain of consulting case interviews in more detail, establishing the conceptual background the rest of the project builds on. The proposed solution follows in Chapter 3: an overview of the architecture, a breakdown of each component, and a discussion of the constraints that shaped the design along the way, including the copyright issues around the source material used for grounding. Chapter 4 turns to evaluation, and

goes well beyond basic unit testing to tackle a harder question: how do you evaluate the behaviour of an AI system that is itself doing the evaluating? Chapter 5 discusses what the experimental results actually mean for the research questions set out above, and Chapter 6 closes the dissertation with its conclusions.

2 Context & Related Work

This chapter reviews the key literature informing the project: consulting, multi-agent systems, interview simulation, and rubric-based evaluation. These areas support the design of an agentic AI system that simulates case interviews, assesses reasoning, and provides structured feedback.

2.1 Consulting Case Interviews

Consulting case interviews are a common format in consulting recruitment. As explained before, in this type of interview, the candidate is given a business situation or problem and has to explore it, ask relevant questions, analyse the information provided, and propose a recommendation. The case can be presented orally, in writing, or through a combination of both [4]. The purpose of these interviews is to assess how the candidate approaches the problem, structures the analysis, communicates their reasoning, and demonstrates business intuition [5].

There are different types of consulting cases, ranging from profitability and market sizing to strategy, market entry, pricing, or operational improvement. In a profitability case, the candidate needs to work out why a company is losing money by breaking down its revenues and costs. In a market sizing case, the candidate estimates the size of a market from a set of reasonable assumptions and back-of-the-envelope calculations.

2.2 Multi-agent systems

A multi-agent system is one in which multiple autonomous entities, each holding a local view of the environment and its own goals, interact – through communication or a shared environment – to produce a behaviour that no single one of them could produce alone [6]. A single-agent system, by contrast, has one process holding the full state and responsible for every function of the task.

With large language models, this contrast reappears in a new form. A single LLM call can be asked to play several roles at once within one prompt – asking, judging, and evaluating in the same pass over the same context – and nothing about doing so requires more than one model instance or any exchange between parties. A multi-agent LLM system, by contrast, restricts each role to its own agent: each one holds only the context its role needs, produces its own output, and that output becomes part of what another role's agent then acts on. Park et al. [7] apply this to believable role-play, where

each LLM-backed persona acts only on its own private memory and observations; Li et al. [8] apply it to collaborative task-solving, where two role-conditioned LLM agents communicate to complete a task neither role's prompt alone would solve. In both cases, the justification for splitting the task across agents rather than one model is the same: each agent's context stays restricted to what its role needs, and each role can be configured and assessed independently of the others.

This is also what separates a multi-agent system from a single LLM. Plaat et al. [9] distinguish three complementary capacities that an agentic LLM system may exhibit: reasoning, where a model's own decision-making is improved, for instance through self-reflection or retrieval, without any second party involved; acting, where a model is given an interface to affect the outside world; and interacting, where distinct agents take on separate roles and communicate to jointly reach an outcome. A single LLM conversation, however elaborate the prompt or however many intermediate steps it takes, stays within the first two capacities: one process reasons and, at most, acts, but there is no second party with its own role and its own stake in the outcome. What the interacting capacity adds, and what distinguishes a multi-agent system from a single agent reasoning about its own output, is precisely the presence of a second, independently-configured agent whose response can redirect the first agent's subsequent behaviour.

2.3 Prior work

Using an LLM to conduct a dynamic, question-driven interview is not a new idea, and several existing systems apply it to job interviews specifically. EZInterviewer is one of the earliest examples. It generates the next interview question conditioned on the candidate's resume, the job description, and the dialog so far, with the explicit goal of making mock interview practice more accessible than relying on peers or paid coaches [3]. SimInterview extends this idea into a business education setting, combining interview simulation with feedback and multilingual support, so that the practice session does not end with a question but continues into some form of learning support [10]. Kim et al. [11] take the idea in a different direction: rather than helping a human practice, they use an LLM interviewer to dynamically evaluate another LLM, replacing a fixed set of static test questions with an adaptive, multi-turn interaction that can ask follow-ups and give feedback in real time.

Taken together, these three examples show that the general concept explored in this dissertation – an LLM that interviews dynamically – is not new. What varies across them is the object of the interview (a job candidate, a business student, or another model) and how much of the interaction is dedicated to producing something useful after the conversation ends. None of them, however, is built around the demands of

a specific technical domain: EZInterviewer and SimInterview are designed to generalize across job roles, and Kim et al. [11] evaluate general reasoning, factuality, and instruction-following rather than domain-specific problem-solving.

Socratic tutoring systems offer a different angle on the same idea of guided interaction. Rather than a system that asks questions to evaluate a candidate, a Socratic system asks questions to teach: it withholds the correct answer and instead uses questions, hints, and targeted feedback to make the learner work through the reasoning themselves. Favero et al. [12] build a Socratic chatbot along these lines to promote critical thinking, which suggests this style of guided questioning has some precedent in LLM-based tutoring, even outside the interview format specifically.

2.4 LLMs as a judge

As stated previously, consulting case interview is assessed based on the quality of a reasoning process, which means the evaluator has to judge *how* the candidate thought, not just *what* they concluded.

LLM-as-a-Judge is the general paradigm this project draws on to address that problem. Instead of comparing an output against a reference string, an LLM is prompted to read the output and produce a judgement against some stated criteria, in a way that is meant to mimic how a human expert would evaluate it. Gu et al. [13] survey this paradigm and show that it is not without problems: LLM judges can disagree with human evaluators, can be swayed by superficial features such as response length or ordering, and can behave inconsistently across repeated runs on the same input.

A judge that simply outputs "this is a good answer" or "this is a weak answer" inherits all of these failure modes with no way to inspect where they came from (there is no way to tell whether a low score reflects a weak structure, a poor assumption, or a communication problem). Gu et al. [13] discuss one way to address this opacity: splitting a single broad criterion into several narrower ones, each scored on its own before the results are combined, rather than asking for one overall verdict.

2.5 Retrieval-Augmented Generation

An LLM's knowledge is fixed at training time and stored implicitly in its parameters, which makes it prone to hallucination whenever a question falls outside what it has reliably learned, or requires up-to-date or specialised knowledge it was never trained on – rather than acknowledging the gap, the model tends to produce a plausible-sounding but ungrounded answer. Retrieval-Augmented Generation (RAG) addresses this by pairing the LLM with an external knowledge source: relevant documents are retrieved at inference time and inserted into the prompt, so that the model's response

is conditioned on concrete, checkable material rather than solely on what it happened to memorize [14].

Grounding an output in retrieved text is only useful if the retrieved text is actually relevant, complete, and correctly matched to the query, and a retriever can just as easily surface material that is tangential, redundant, or missing the one passage that mattered [14]. An LLM conditioned on the wrong documents tends to produce a fluent answer that is grounded in the wrong thing, which is arguably worse than an answer that is visibly unsupported, since it looks credible without being reliable.

In practice, RAG is implemented as a pipeline with two stages [14]. In an indexing stage, the external corpus is split into smaller *chunks* - since embedding and retrieving an entire document is both computationally wasteful and imprecise - each of which is mapped to a vector representation by an embedding model and stored in a vector database for similarity search; chunks are typically given some *overlap* with their neighbours so that content near a chunk boundary is not split away from the context it needs to be understood. At inference time, the query is embedded with the same model and used to retrieve the most similar chunks, which are then inserted into the LLM's prompt alongside the query itself.

2.6 Research Gap

The literature reviewed in this chapter shows that the necessary building blocks already exist, just not combined and not applied to this domain. LLMs can conduct dynamic, multi-turn interviews rather than follow a fixed script [11]; they can generate interview questions grounded in a candidate's background [3]; they can judge open-ended responses, provided the judgement is decomposed into explicit dimensions rather than a single verdict; and they can be grounded in external material through retrieval rather than relying only on their internal knowledge [14]. What is missing is their combination, applied to consulting case interviews specifically: existing tools either ask good questions without judging the answer, or judge a response without conducting the interaction that produced it, and none of them target this domain, which leaves no public dataset of case interviews to train or benchmark against either.

This project addresses both gaps. Architecturally, it separates interviewing, judging, and feedback into coordinated agents, and evaluates the resulting specialised multi-agent architecture against a single-agent baseline. In place of labelled training data, it grounds questioning and evaluation in theoretical guidance through RAG, judging candidates against predefined rubric dimensions.

3 Proposed Solution

3.1 Overview

The solution built for this dissertation is a single codebase that runs two different graphs on the same inputs, a baseline graph and an agentic graph. Neither is a separate application. They share the same loader, the same case and scenario data, and the same rubric, and differ only in how the interview, evaluation, and feedback responsibilities are wired together internally.

Two tools make the system's internal behaviour inspectable. LangSmith was used throughout development to trace a single run turn by turn and see exactly what each node read from the state and what it wrote back (§3.3). A small workbench web page was also built to look at experiments at a higher level – browsing transcripts, RAG retrievals, and per-agent scores across many runs at once. Most of the qualitative conclusions in this dissertation came from that workbench.

Interviewer, **Judge**, **Feedback**, and **Baseline** are the agents proposed by this system. The **Candidate** is not one of them as it is an LLM-simulated proxy for a human interviewee, for reasons discussed in §3.4.3. At the judge level, evaluation is split into two parallel nodes, *Evaluator of Case Performance* and *Evaluator of Dialog Quality*, instead of one combined call.

Two runnable systems sit side by side on identical inputs:

- **Baseline**: a single-agent design in which one node plays interviewer, evaluator, and feedback-writer in one combined LLM call per turn.
- **Agentic**: a multi-agent design in which an Interviewer, a Candidate, a Judge, two specialised evaluation nodes, and a Feedback agent each have their own prompt, their own turn, and their own share of the shared state.

It is tempting to picture an agent as nothing more than a prompt, one call to an LLM, and whatever text comes back. That is not quite what happens here, because every agent, baseline or agentic, writes into a shared *state* instead of just producing a message. Different fields serve different purposes – one holds the agent's own internal reasoning, another holds a query to the knowledge base when it needs to retrieve something – and populating more than one of them can mean several calls to the underlying model in a single turn, not just one. The two graphs don't carry the same state either; each only holds the fields its own agents need, described in full in §3.3.2.

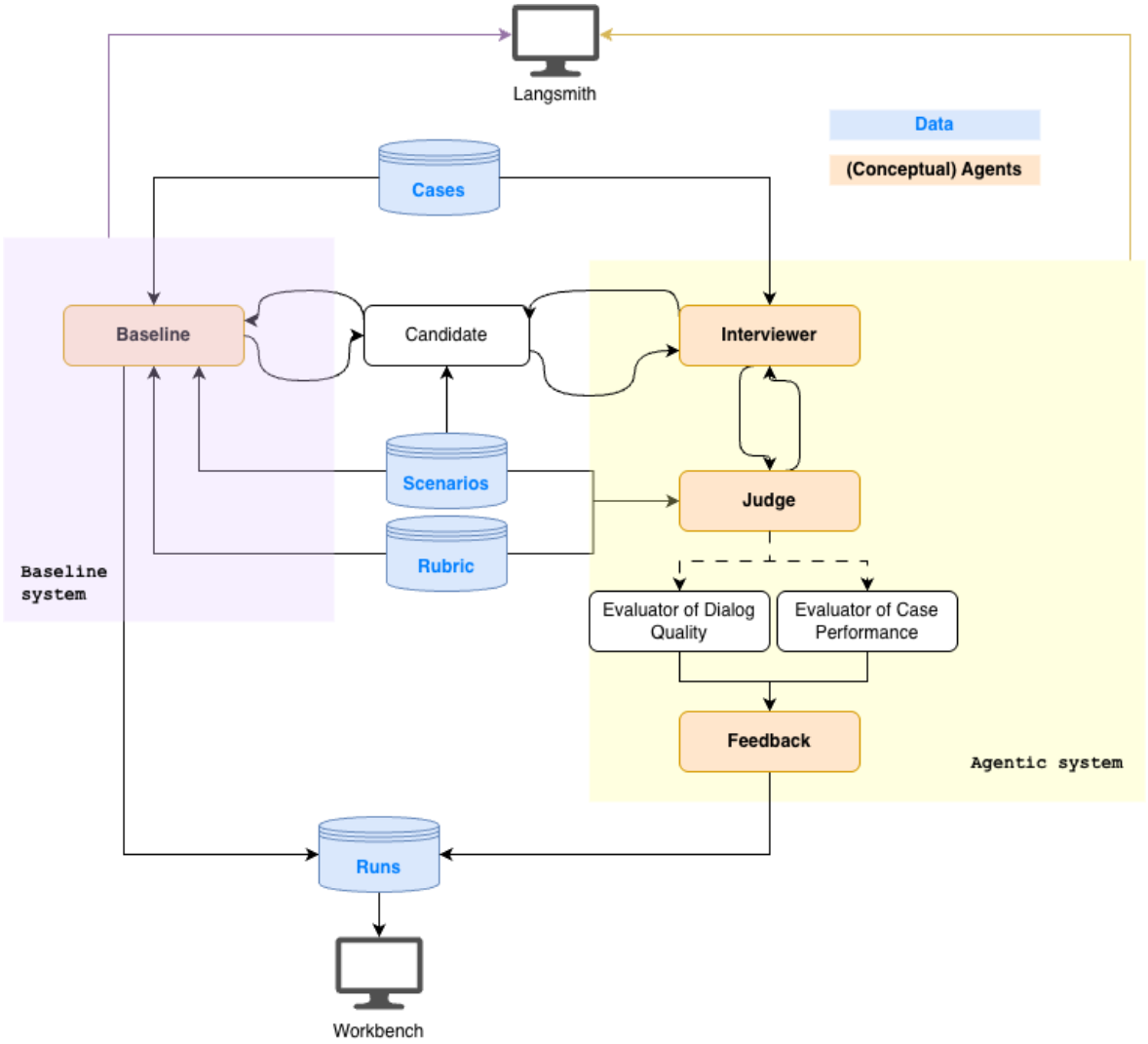


Figure 3.1: High-level view of the two-graph system.

3.1.1 Baseline and Agentic comparison

Before comparing the two graphs directly, it is worth distinguishing a *node* from the *conceptual agent*. A node, in the LangGraph sense used throughout this chapter, is a unit of execution that performs a computation and returns an update to the graph state (such as an LLM call, a Python function, or an HTTP request). A conceptual agent – Interviewer, Feedback, Judge, Candidate – is a *role*, meaning a persona with its own information access and its own decision to make. In the agentic graph, the two do not always coincide. The judge has three nodes whereas the Interviewer only has one node, explained in §3.2.5. Table 3.1 summarises the resulting structural differences.

Table 3.1: Structural differences between the baseline and agentic graphs.

Dimension	**Baseline**	**Agentic**
Total nodes	4	8
Interviewing, judging, feedback	One combined LLM call per turn, one unified schema	Separate nodes, each with its own prompt, LLM call, and turn
Judge independence	None (Interviewer's own first-pass opinion, never checked)	Independent second call, can send case back for another round
Case-guide PDF retrieval	LLM decides opportunistically inside the same combined call; excerpt only available the *next* turn	Separate LLM scouting call in `judge`, `eval_case_performance`, `eval_dialog_quality` and `give_feedback`; excerpt used the *same* turn. Never available to the Interviewer while it is questioning the candidate
Profitability-guide PDF retrieval	LLM decides opportunistically inside the same combined call; excerpt only available the *next* turn	Separate LLM scouting call, restricted to `eval_case_performance` only; excerpt used the *same* turn. Not available to the Interviewer, Judge, dialog-quality evaluation, or feedback nodes
Evaluation output	Combined payload from the stopping call	Two parallel, specialised evaluation nodes
System prompt	One standalone, general-purpose persona	Composed per role from separate files

Note. "Scouting call" is defined in full in §3.2.3; in short, it is a dedicated call a node makes to its own role LLM purely to decide whether it needs a knowledge-base excerpt right now.

3.2 Architecture

This section describes the system built to test the comparison introduced in the previous chapter. It proceeds bottom-up: the Data subsection first covers how the casebooks were used to build the synthetic cases and scenarios both systems consume, and the copyright constraint that shapes how they were built; the Model Selection subsection covers which models were chosen for each role; the RAG System subsection covers the retrieval mechanism shared by both graphs. With that common ground in place, the Baseline System and Agentic System subsections describe the two graphs themselves – what each collapses or separates, which node owns which decision, and the design choices specific to each.

3.2.1 Data

This section differentiates the data sources consumed by the system. First, the Interviewer agent consumes the consulting cases (what an interviewer would present to a candidate). The candidate agent consumes the scenario, which defines the candidate's behaviour so as to simulate a real case. In addition, to judge the interview correctly, the system relies on a scoring rubric[1] and is fed with certain PDFs as a knowledge base to answer with greater accuracy. Both systems consume exactly the same information.

The following diagram gives a high-level overview of each of these sources.

[1] `https://assets.managementconsulted.com/app/uploads/2025/06/02180347/Management-Consulted-Case-Interview-Scoring-Guide-2025.pdf`

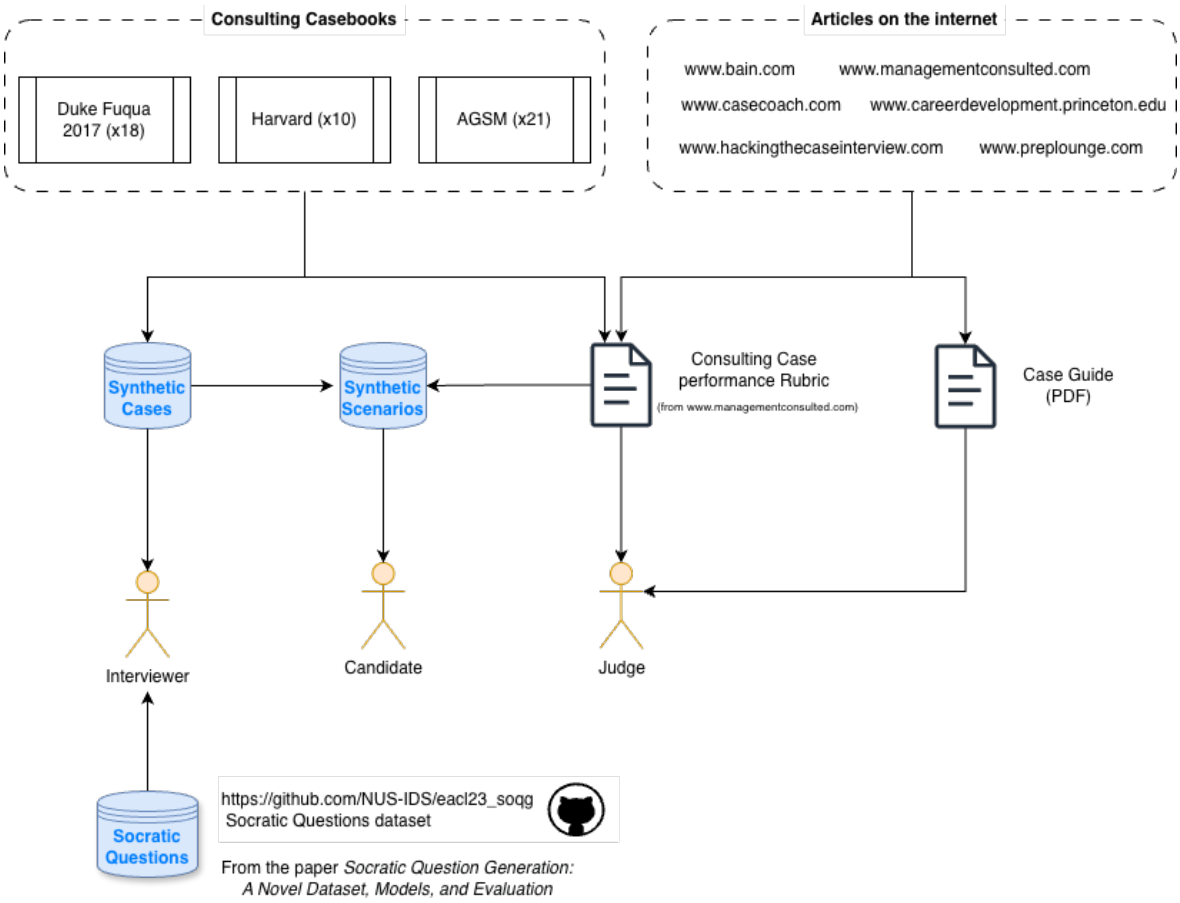


Figure 3.2: High-level view of the data

Casebooks

The synthetic consulting cases developed in this project were structurally based[2] on three well-known casebooks–the AGSM casebook [15], the Duke Fuqua 2017 casebook [16], and the Harvard casebook [17]. All three were obtained from a public repository of case interview examples Career in Consulting [18]. This repository also served as the primary reference for understanding the general theory and common structure of profitability cases. To narrow the scope of the project, only profitability cases were selected, specifically cases in which a client reports declining profit, flat profit, or margin pressure, and the candidate is expected to diagnose the underlying driver and recommend an appropriate course of action. Each case in the corpus follows the same basic structure, with an opening prompt describing the client situation, a set of data initially provided by the interviewer, additional information revealed only if the candidate explicitly requests it, and guidance describing the expected response.

Despite differences in industry and difficulty, almost every profitability case in the corpus follows the same underlying logic. Profitability, in this context, means how

[2] "Based on" refers to the shared structural pattern, not to reuse of the casebooks' text; no casebook content was copied (§3.4.1).

efficiently a company converts revenue into profit – typically net income as a percentage of sales Goedl [19] – and a profitability case asks the candidate to explain why that percentage has changed. Each case opens with a puzzle in which a client's profit has either declined outright or stayed flat while some other part of the business, such as revenue or market share, moved in a different direction. From there, the candidate structures the problem into a diagnosis and narrows it down to a single dominant driver – cost inflation, an unfavourable shift in product or channel mix, underused capacity, or price pressure – supported by a small number of decisive facts revealed progressively rather than upfront. The final step converts that diagnosis into an action recommendation covering the answer, its rationale, the main risks, and what to test next if uncertainty remains.

Synthetic Cases

Building on the casebooks and the common structural pattern identified in the previous section, three cases were created around deliberately unrelated topics: an energy company (Solventus Energy), a football club (CF Baluarte Levante), and an agri-tech startup (Vertex AI). A fourth case, built around the FIFA World Cup expanding from 32 to 48 national teams, was created separately and used exclusively for testing. Each case is stored as a JSON file composed of typed blocks – `prompt`, `guidance`, `math`, `data`, `expected_analysis`, and `final_recommendation` – and every block carries a `visible_to_candidate` flag, so the agent acting as the interviewer (either the Interviewer or the Baseline agent) always knows what information it may reveal to the candidate at any given point.

These cases were first drafted creatively, drawing on the pattern observed across the profitability cases reviewed earlier, and were then refined using a commercial AI tool (OpenAI's ChatGPT, GPT-5) to correct spelling errors, convert the content into valid JSON, and check internal consistency. The rationale for using synthetic rather than real casebook-derived cases is discussed in Section 3.4.1.

Synthetic Scenarios

Each scenario scripts the candidate agent's behaviour for one synthetic case. As shown in Figure 3.3, `candidate_profile` nests two blocks: `persona_instruction` (`role`, `behaviour_description`, `behavioural_rules`, plus `case_specific_facts`, `solution_roadmap` and `math_guidance` to anchor the persona to the case rather than leave it improvised) and `reference_scores`, which grades every rubric and `case_interaction_quality` dimension on the same 1–4 scale, marking untested dimensions (e.g. `case_math_answer` in a qualitative case) as `not_tested`.

Each case has eight to nine freely-authored scenarios, with `case_specific_facts`

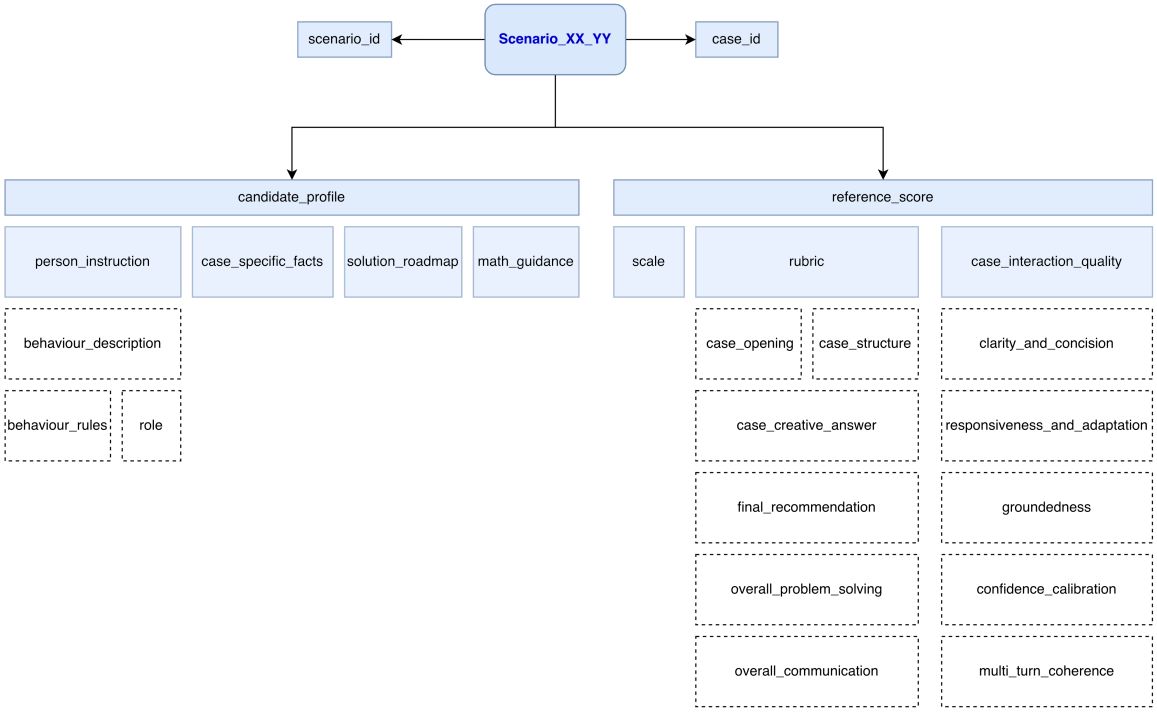


Figure 3.3: Schema of a profile given as a scenario

and `solution_roadmap` kept in sync with the case JSON to avoid stale data. These personas isolate a single flaw against otherwise strong performance, so reference scores cluster at the extremes (1 on the flawed dimension, 4 elsewhere), leaving the middle of the scale – a reference score of 2 – essentially untested. To close this gap, each case was extended with sixteen dimension-targeted scenarios: one `_l3`/`_l2` pair per gradable dimension (`case_opening`, `case_structure`, `final_recommendation`, `clarity_and_concision`, `responsiveness_and_adaptation`, `groundedness`, `confidence_calibration`, `multi_turn_cohere`
`_l3` isolates the dimension at `expected`=3 with everything else at 4; `_l2` drops it to 2, with a correlated `overall_*` dimension falling to 3, so the judge is exercised on both mid-scale scores. This adds 48 scenarios (16 per case) to the original 26, for 74 in total. As with the cases, scenarios were drafted creatively and refined with a commercial AI tool for JSON validity and internal consistency.

3.2.2 Model Selection

Table 3.2 shows the final model assigned to each role. Role requirements drove the choice rather than raw benchmark scores, including architectural diversity between Judge and evaluated roles [13], roleplay consistency for the Candidate [20], and context-window limits for Feedback. The full justification is in Appendix A.

Table 3.2: Final model configuration by role.

Role	Model	Temp.	Primary strength
Interviewer	Llama-3.3-70B	0.6	Dynamic multi-turn question generation and instruction following.
Judge	Qwen2-72B	0.0	Low-bias evaluation and score calibration.
Candidate	Gemma-3-27B	0.7	Multi-turn consistency, reasoning, and long-context support.
Feedback	Llama-3.3-70B	0.3	Coherent long-form feedback generation.
Baseline	Llama-3.3-70B	0.5	Strong general-purpose performance across all roles.

3.2.3 RAG System

Two independent retrieval-augmented generation paths ground the system's outputs in external methodology, following Gao et al. [14]: a case-guide source, the internal consulting case-interview guide compiled from personal notes taken while learning about consultancy case interviews, and a profitability-guide source, Goedl [19]'s managerial accounting textbook. Each is embedded once into its own persistent Chroma collection with a local `FastEmbedEmbeddings` model (`BAAI/bge-small-en-v1.5`), split into 900-character chunks with a 120-character overlap. Neither source depends on a network call at query time, and a chunk can always be traced back to a page number for citation. The two sources stay in separate collections because they answer different questions – interview methodology and structuring versus quantitative accounting logic – and every retrieved chunk in the transcript and workbench is tagged with which one it came from.

Retrieval is a decision each node makes for itself, not something a shared "RAG node" issues on the rest of the graph's behalf. Every RAG-capable node builds the same situation context it would use for its main call anyway, asks its own role LLM whether it needs an excerpt from a given source right now, and only retrieves if that scouting call returns a non-empty query. An empty query means retrieval is skipped for that node this turn. Table 3.3 lists which node consults which source, and how quickly the excerpt becomes available to it.

Table 3.3: Which nodes consult which RAG source, and when the excerpt they ask for becomes available.

Node	**Case guide**	**Profitability guide**	**Timing**
Interviewer	-	-	Never consulted.
Candidate	-	-	Never consulted.
Judge	Own scouting call	-	Same turn: excerpt is retrieved and shown before the Judge's own evidence-sufficiency call.
Judge (Eval. Case Performance)	Own scouting call (combined)	Own scouting call (combined)	Same turn: one call can request either or both sources before scoring.
Judge (Eval. Dialog Quality)	Own scouting call	-	Same turn.
Feedback	Own scouting call	-	Same turn.
Baseline	Opportunistic, inside the single combined call	Opportunistic, inside the single combined call	*Next* turn: a query that writes this turn is only resolved and shown back to it on the following turn.

The baseline's retrieval is the one deliberate asymmetry between the two graphs. Collapsing the roles (§3.1.1) leaves it no spare turn for a dedicated scouting call, so it can only flag a retrieval need opportunistically while producing that turn's output, and the request resolves only on the following turn. This lag introduces an additional architectural difference between the baseline and agentic systems, namely same-turn, LLM-scouted retrieval versus a delayed, opportunistic equivalent.

3.2.4 Baseline System

This baseline is the control condition against which the agentic graph is measured. A single node assumes all roles at once, and this collapse is deliberate, providing a practical single-agent control against which the complete specialised multi-agent architecture can be compared.

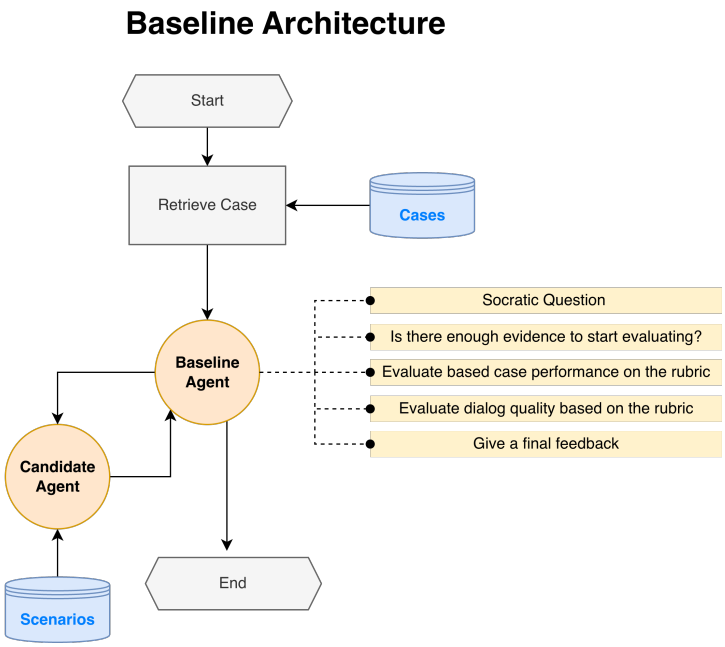


Figure 3.4: Baseline Graph Architecture

Nodes and State

The baseline graph has two nodes that matter for the interview loop, namely `baseline`, carrying the collapsed role described above, and `candidate`, the same LLM-simulated interviewee used by the agentic graph (§3.4.3).

The first turn is not a decision made by the model. The opening line shown to the candidate is the case's prompt, reproduced as-is, so the node's first decision is always a reaction to the candidate's first response, never a cold start.

From that point on, every turn resolves in a single LLM call what the agentic graph keeps as separate calls, including deciding what to say next to the candidate, deciding whether the interview is ready to move to evaluation, and, once it is, producing the evaluation and the feedback text together. One call, one turn, no intermediate step.

Design Rationale

Collapsing the roles into one node has three consequences for how the baseline behaves:

- The decision that the interview is ready for evaluation is the node's own self-assessment; no independent Judge call can contradict it. A single hard turn limit is enough on its own to guarantee the loop terminates, since there is no separate evaluation round to defer the stopping decision to.
- The node runs a single, general-purpose system prompt, not one composed from separate role-specific files.

- Retrieval follows the same logic (§3.2.3): the baseline has no dedicated scouting call for either RAG source, so a retrieval need can only be flagged opportunistically inside the same combined call.

3.2.5 Agentic System

Here, different roles are carried out by different agents. Figure 3.5 shows the resulting graph, consisting of six independent LLM calls – Interviewer, Candidate, Judge, the two parallel evaluation nodes, and Feedback – each with its own prompt and its own turn. No prompt is reused across responsibilities.

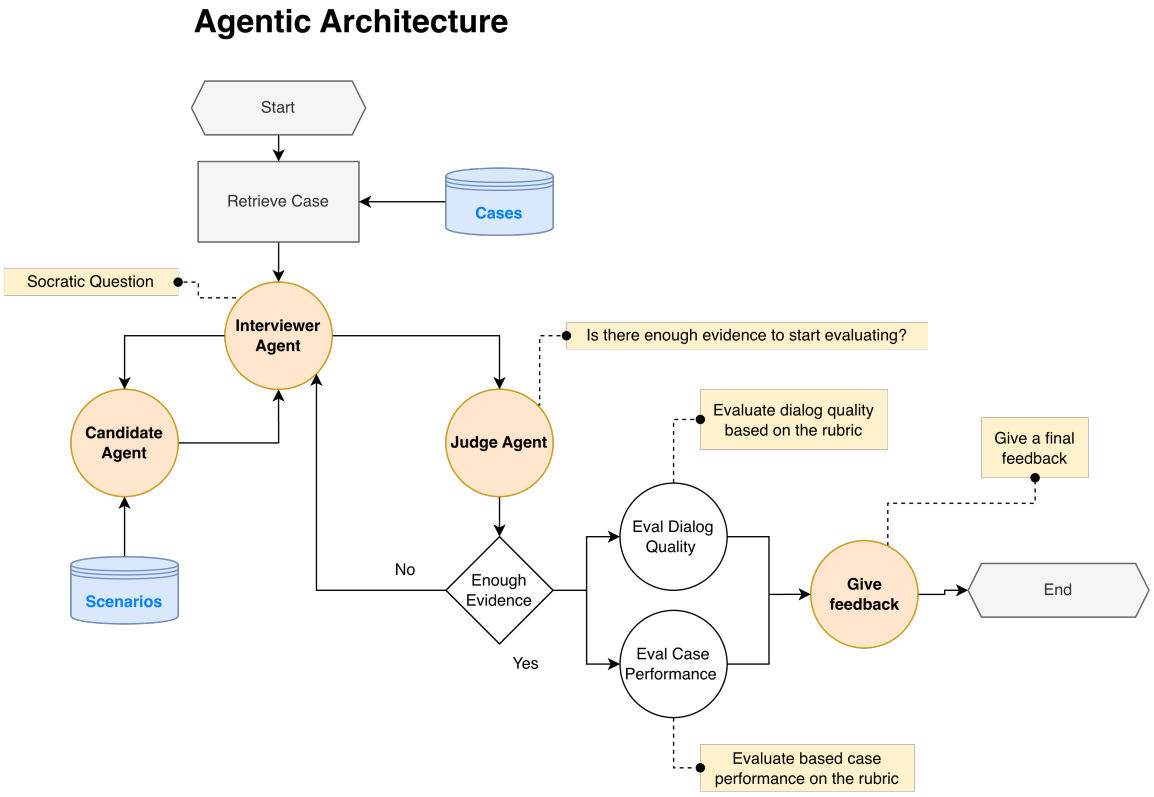


Figure 3.5: Agentic Graph Architecture

Nodes and State

The Interviewer and Candidate exchange turns exactly as in the baseline, but the Interviewer's role stops at asking the next question; it does not decide when the interview is over. That decision belongs to the Judge, consulted once the Interviewer has used up its own turn budget, which can either agree that there is enough evidence to evaluate or send the case back for another round. Only once the Judge is satisfied do the two evaluation nodes run in parallel, each scoring a different part of the rubric from the same transcript. Their outputs then pass to a dedicated feedback node, which produces the interview's final write-up.

Design Rationale

The Judge's independence is the central design decision this graph is built around. It can disagree with the Interviewer's own sense that the interview is ready, and send the case back for another round with concrete guidance on what still needs to be probed – not a generic instruction to keep going. This is the direct counterpart to the baseline's self-assessment, and it is also why the loop needs two turn budgets instead of one, with a limit on how many turns the Interviewer gets before the Judge is consulted and a separate limit on how many rounds the Judge itself can request before evaluation is forced. The Judge's guidance is read back by the Interviewer on the next round and shapes what it actually asks next; that is the mechanism through which another round actually changes the interview, instead of just repeating it. Because a Candidate's answer to the Interviewer's very last question must never be allowed to bypass evaluation, the graph also enforces a deterministic safeguard on the final forced turn, so the loop cannot terminate on the model's own say-so at the one point where that would matter.

`Eval Case Performance` and `Eval Dialog Quality` are, conceptually, still the Judge, since both score the same judge-approved transcript against the same rubric, and neither exists as an independent point of view in the way the Judge does relative to the Interviewer. The split is technical. The Judge's evaluation role runs as three nodes so the two halves of the rubric can be scored in parallel rather than serially, echoing the node-versus-agent distinction from §3.1.1. Feedback, by contrast, is a genuinely separate role, as it has its own prompt and its own call and does not share in the Judge's evaluation the way the two scoring nodes do.

Each of these six roles is composed from its own prompt, which is what allows one role to be tuned without touching the others. The same separation carries over to what each role is allowed to see, since the Interviewer never has direct access to the Judge's own reasoning and only sees what the Judge decides to feed back into the next question. Retrieval also resolves within the same turn for every RAG-capable node here, unlike the one-turn lag the baseline's collapsed call introduces.

3.3 Technical Implementation

The full implementation is available on GitHub.[3]. This section covers how the two graphs are actually built, including why LangGraph was chosen over a hand-rolled loop and how each node and role's state is laid out. Prompt engineering, the engineering challenges encountered along the way, the persistence layer, the workbench, and the overall code base organisation are documented in Appendix A

[3] `https://github.com/PPerezdeMadrid/agentic-case-interview-coach`

3.3.1 Why LangGraph

A simple loop of LLM calls would have been enough for a linear pipeline. TThis system is not linear, as it involves multi-turn interactions between the Interviewer and Candidate, judge-triggered retries, and concurrent evaluation of the same Judge decision. Passing raw strings or dictionaries between calls quickly becomes unmanageable, so the graph is built around a single typed state object (`AgenticGraphState`, §3.8) that every node reads from and partially updates. Each field defines how partial updates are merged, overwriting by default or using a custom **reducer** when a different strategy, such as appending, is required. The two parallel evaluators rely on custom reducers for `rag_query_log` and `llm_usage` – otherwise one branch's writes would overwrite the other's.

Routing between nodes is handled by plain Python functions that return an edge name (`route_after_interviewer`, `route_after_judge_agentic_02`), not another LLM call, so turn and round budgets are enforced deterministically and the system never exceeds its configured limits. The two evaluation nodes are declared as a genuine fan-out from the Judge (`add_edge(["eval_case_performance", "eval_dialog_quality"], "give_feedback")`), so LangGraph runs them in parallel and waits for both to finish before moving on – no manual synchronisation needed.

LangSmith handles tracing by plugging into the LangChain/LangGraph stack with no extra instrumentation, unlike a separate tool such as Langfuse [21]. Being able to inspect every execution step – what state each node read and wrote – turned out to be crucial for catching design flaws that weren't visible from the final transcript. Several of the engineering challenges in Appendix A.0.2 were found this way.

3.3.2 States and Node Construction

Both graphs share a common state (Figure 3.6) holding everything that does not depend on which pipeline is running, including the case prompt, the candidate profile, the transcript, the case guidance and recommendation, the running dialog quality and case performance scores, and control fields such as `turn_index`, `thread_id`, or `run_id`. Each graph then extends this base with only the fields it needs.. The baseline adds `pending_case_guide_query` and `pending_profitability_query`, since a single agent decides in one shot whether to retrieve supporting material. The agentic graph adds `focus_areas`, `judge_round`, and `candidate_reasoning`, reflecting an interview broken into rounds coordinated by a dedicated Judge.

Figures 3.7 and 3.8 show how these state slices map onto concrete Pydantic submodels. In the baseline graph, a single `Baseline Agent` node writes into `BaselineTurnOutput`, grouping the Interviewer's move (`reasoning`, `action`, `content`, `block_id`), its RAG decision (`case_guide_query`, `profitability_query`), and the evaluation it produces

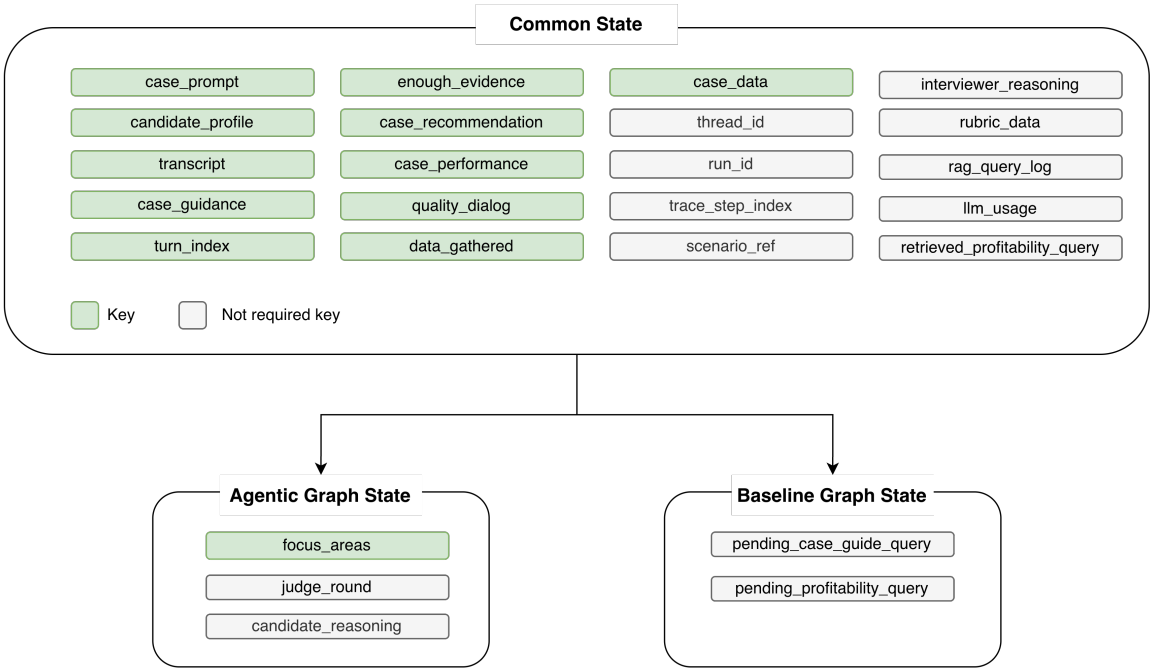


Figure 3.6: Common State (Fields both graphs read/write identically)

once the conversation is ready (`case_performance`, `quality_dialog`, `feedback`). The agentic graph splits these same responsibilities across three conceptual roles – `Interviewer`, `Candidate`, and `Judge` – each with its own schema (`InterviewerMove`, `CandidateResponse`, `JudgeResponse`).

Most nodes follow the same shape:

1. Build the LLM messages from the current state.
2. Optionally run RAG scouting (§3.2.3).
3. Call the role's LLM through the shared helper `invoke_json_llm`, which retries on malformed JSON up to N times.
4. Write the parsed result back into the state.

Each node's message-assembly step is a pure function, with no side effects and no dependency on anything beyond the state it is given. That is what later made it possible to build the node-level evaluation golden sets directly against the real, production message-building code (§4.1.3), avoiding a hand-copied approximation prone to drifting out of sync.

The feedback node is the one real exception as it skips `invoke_json_llm` entirely. Feedback is free-form prose, not a JSON schema, so it calls the underlying chat model directly and extracts the text response.

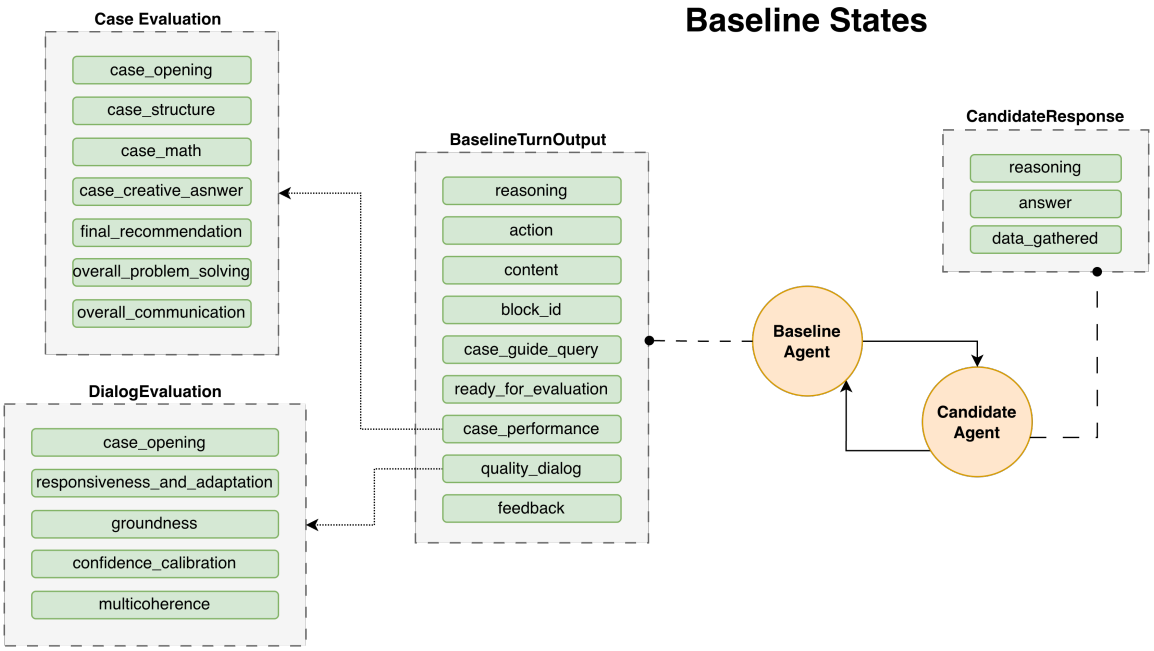


Figure 3.7: LangGraph States from Baseline Graph

3.4 System Constraints

3.4.1 Copyright

The original design also included two agents for generating and validating candidate consulting cases, so that the entire practice ecosystem would be AI-generated. Work started from reading actual profitability cases and the theory behind them, drawn from university career-services casebooks (AGSM [15], Duke Fuqua 2017 [16], Harvard [17]) and from consultancy firms' own published cases.

These casebooks carry explicit copyright notices reserving all rights to their respective institutions. Under s.29A of the Copyright, Designs and Patents Act 1988, a person with lawful access to a work may copy it without infringing copyright, provided the copy is made "for the sole purpose of research for a non-commercial purpose" and is accompanied by sufficient acknowledgement.[4] This supports reading and analysing the casebooks to learn the shape of a profitability case, but it is unclear whether a language model consuming that material qualifies as such research. Given the Harvard casebook's explicit copyright reservation, independently flagged as "potentially problematic" by the Copyright team, the casebooks were excluded from direct use in the system.

A second option, scraping live cases from consulting firms' own websites (e.g.,

[4] `https://www.legislation.gov.uk/ukpga/1988/48/section/29A`

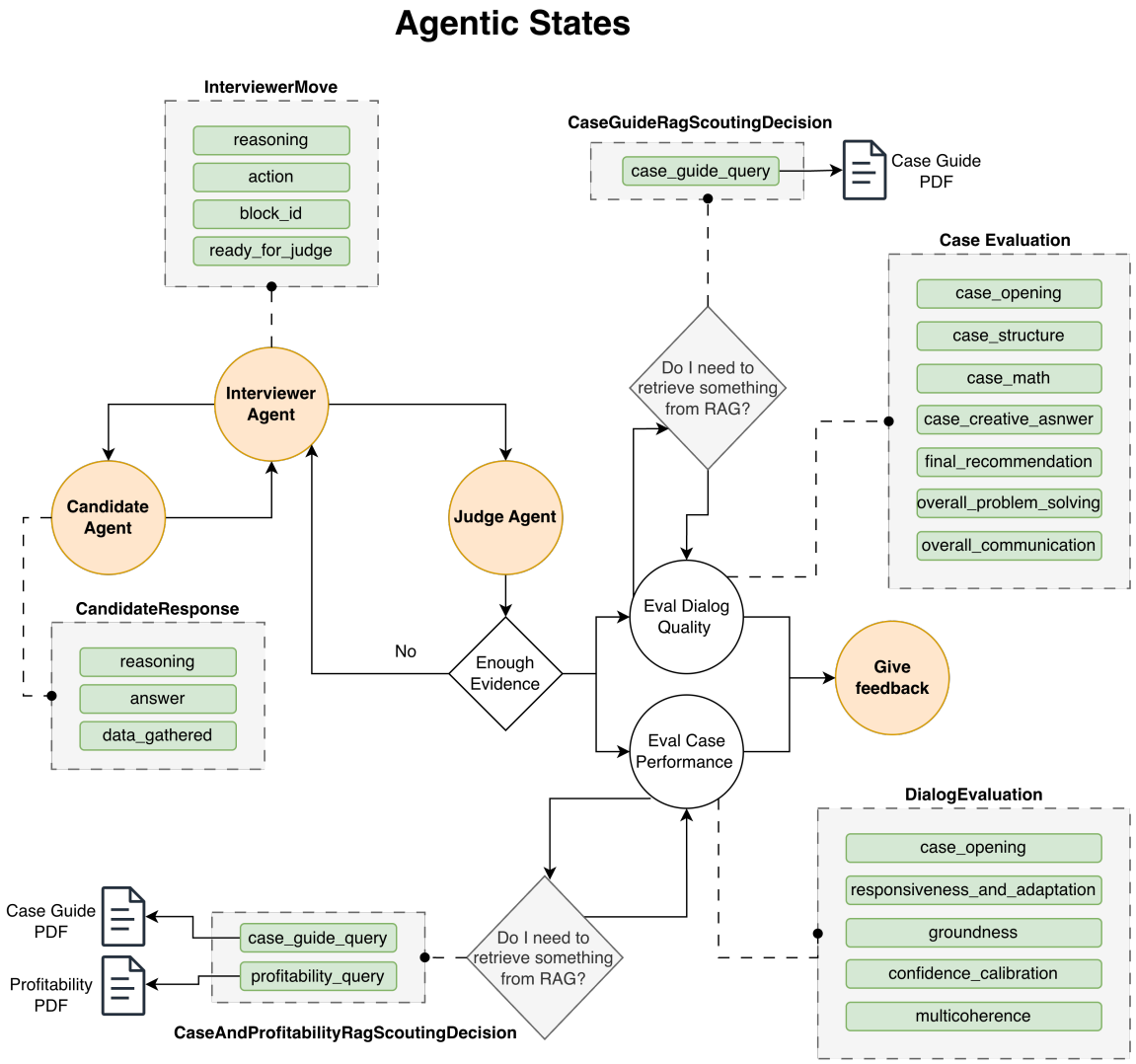


Figure 3.8: LangGraph States from Agentic Graph

McKinsey's *electrolight* interview page[5]), was ruled out because these sites' terms of use restrict content from being used for AI training.[6]

A third option, searching for case material released under an open licence such as CC0 or CC BY, did not yield a suitable independent corpus; most case-interview websites aggregate or link to material originating from the same casebooks and firm publications.[7]

The approach adopted was to extract the structural pattern common to profitability cases, described in §3.2.1, without reusing any casebook content directly. The casebooks' only remaining role is as the source of a compiled internal case-guide used for retrieval (§3.2.3), consisting of personal notes written while learning the material rather than excerpts of the source PDFs. Goedl [19]'s managerial accounting textbook was also used as a RAG source, since it is released under a Creative Commons Attribution 4.0 licence. It is not a perfect fit for consulting interview content, but it is useful for grounding the management accounting concepts that appear across profitability cases.

3.4.2 LLM limitations

Every agent in this system is an LLM, so the limitations of LLM-based evaluation apply throughout, not just at the Judge. Gu et al. [13] and Kim et al. [11] document the main ones:

- **Self-enhancement bias**: an LLM judge tends to score the outputs more favorably produced by its own underlying model [13].
- **Verbosity and concreteness bias**: longer and more confident answers (full of numbers and specific terms) tend to score higher independently of whether the reasoning behind them is actually correct [13].
- **Low self-consistency**: re-running the same evaluation can produce different scores, and Kim et al. [11] report that this variance grows with harder problems. `run_all_scenarios.py` deals with this directly by repeating every scenario on both graphs (`-repeat`, default 4) instead of trusting a single pass. The comparison is designed around this limitation.
- **Error accumulation in agentic pipelines**: Gu et al. [13] note that chaining LLM calls lets an early misjudgment propagate downstream. This cuts both ways for this project; the agentic graph has more nodes for an error to travel through, but also more separate checkpoints where it could be caught.

[5] `https://www.mckinsey.com/careers/interviewing/electrolight`
[6] `https://www.mckinsey.com/terms-of-use`
[7] `https://www.craftingcases.com/case-interview-practice/`

- **Same-family correlation bias**: beyond the judge-baseline overlap addressed above, this configuration also has the Interviewer and Feedback drawn from the same model family, both Llama-3.3-70B (Table 3.2). Since feedback does not score or judge another role's output, this overlap does not implicate Gu et al. [13]'s argument for architectural distinctness the way judge-baseline overlap would. The concern here is narrower: the feedback text may simply inherit the interviewer's own phrasing habits or blind spots instead of reading as an independent pass over the transcript. None of the roleplay or LLM-as-a-judge literature reviewed for this project quantifies this specific effect, so it stands here as an unmitigated limitation, not a resolved one. The Candidate, Gemma-3-27B, sits in a distinct architecture family from every other role, so this bias does not extend to the interviewer-candidate exchange itself.

Human evaluation is not a clean substitute for these problems. Hashemi et al. [22] find substantial disagreement between human judges themselves, and show that raw LLM judgments correlate weakly with any individual human's ratings unless calibrated per-judge. In practice, this means the rubric-based Judge in this system is a best-effort automated proxy, not a ground truth, and its output should be read with that caveat throughout the results.

3.4.3 Candidate as an LLM

Ideally, the candidate role would be filled by real interviewees, and the two graphs compared on how well they support an actual human going through a case. Recruiting subjects, running sessions, and collecting consistent transcripts across both systems was not something the project's timeline could absorb without displacing the comparison it was actually built to make. The candidate is therefore an LLM-simulated proxy, scripted per case through the persona mechanism described in §3.2.1. A user study validating this design choice with real interviewees is proposed in Appendix A

This substitution is a deliberate trade-off. A scripted LLM allows an initial, controlled read on the baseline and agentic comparison, replaying identical personas and identical cases across both graphs. Validating that read against real interviewees remains future work. Hashemi et al. [22] offer empirical support for this approach. In their evaluation pipeline for information-seeking dialog systems, an LLM played the user instead of a human and generated the synthetic conversations. Human judges who assessed whether the transcripts resembled genuine user-assistant conversations agreed 98.8% of the time. The evaluation also went beyond plausibility. A calibration model trained exclusively on conversations with the simulated user generalised to real human-user conversations, predicting human judges' satisfaction ratings after converging with only 80% of the synthetic training data. These findings lend support to the

approach adopted here.

This design also has limitations. Achieving a plausible interviewee required several iterations of the persona prompt, which produced the current three-field structure (`behaviour_description`, `behavioural_rules`, and the score-derived paragraph in `utils.format_candidate_persona`, §3.2.1). Even with this structure, the candidate may still drift from its assigned persona towards the underlying model's default behaviour, a known limitation of LLM role-playing that §3.4.2 discusses.

4 Evaluation

A closer match to the reference score doesn't mean a system is better — it simply means it scores the transcript well. What matters is the system as a whole, whether it prompts the candidate with Socratic questions, reveals information at the right moments, grounds its scoring in the rubric, uses its internal reasoning, and, above all, helps the student improve at facing these interviews.

Evaluation runs across five levels (Figure 4.1), namely agent (an isolated node against a golden set), run (a single transcript), scenario (variability across repeated runs of the same input), experiment (aggregation across all scenarios), and RAG (usefulness of the retrieval step).

Across these five levels, the results fall into two patterns that Chapter 5 develops in full. The first is a task-dependence pattern, in which the agentic system outperforms the baseline specifically on tasks that require tracking evidence across multiple turns, while its advantage narrows or reverses on tasks that depend on a single-turn judgement. The second is a calibration effect induced by RAG, whereby retrieval improves the Judge's numerical accuracy and confidence calibration while also shifting its standard for conciseness even though the underlying transcript remains unchanged. The sections below present these results level by level before Chapter 5 brings them together around these two patterns.

4.1 Agents

4.1.1 Evaluation Dimensions

For each situation, the Interviewer must pick the correct action from a set of mutually exclusive labels – the same situation-to-label mapping that underpins the golden set in §4.1.3. I evaluate four dimensions (Figure 4.2):

The Judge instead answers "is there enough evidence to start evaluating?" and I assess it from two angles (Figure 4.3). Quantitatively, I measure the verdict's accuracy, precision and recall. Qualitatively, I classify *why* it gets it right or wrong. A `True` splits into clean coverage, coverage with a messy process, coverage with some content flaw, or coverage with hallucinated data. A `False` splits into incomplete coverage, unfinished analysis, premature conclusion, a non-responsive answer, or a misreading of the evidence already provided.

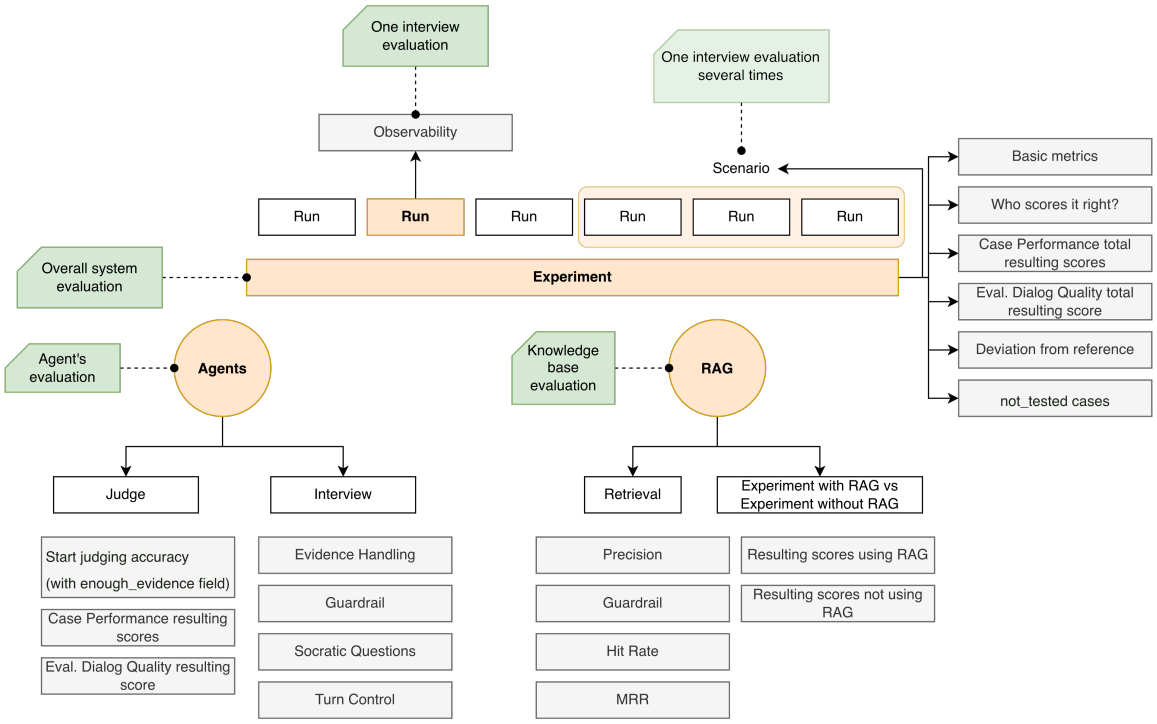


Figure 4.1: Overview of the evaluation pipeline.

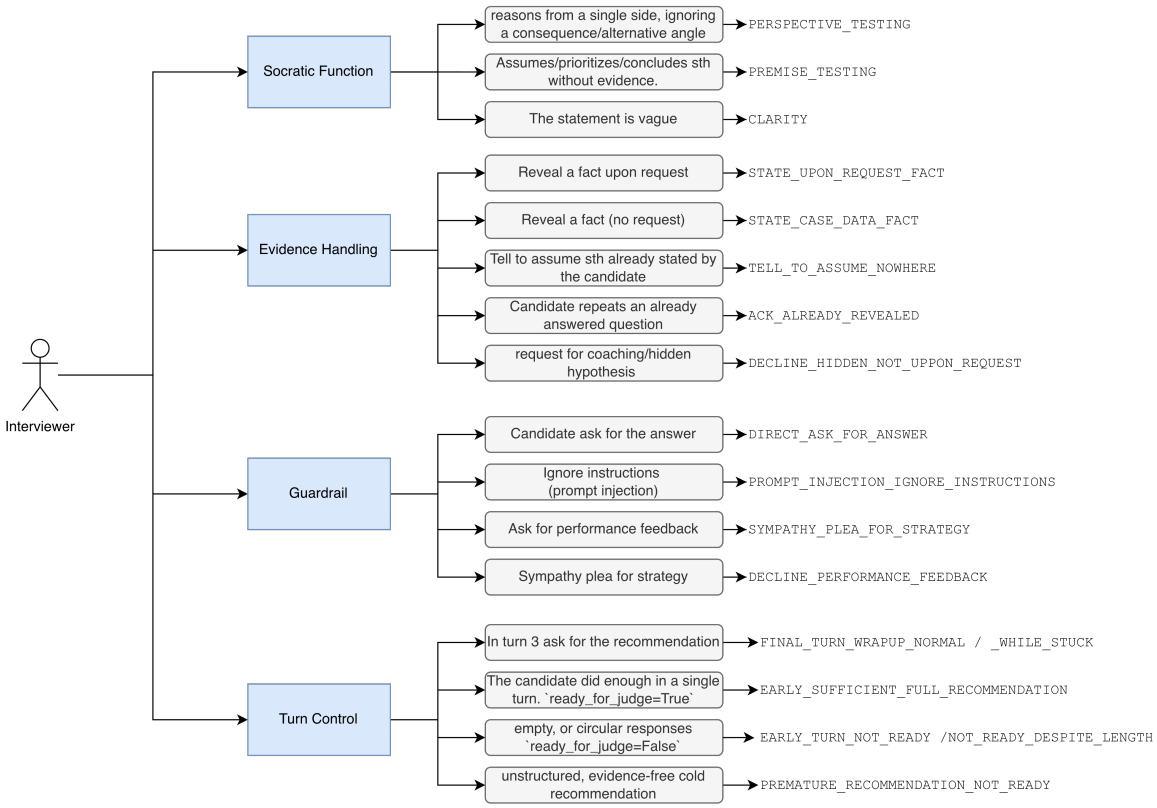


Figure 4.2: Interviewer evaluation taxonomy: conversation situation mapped to the expected label, by category.

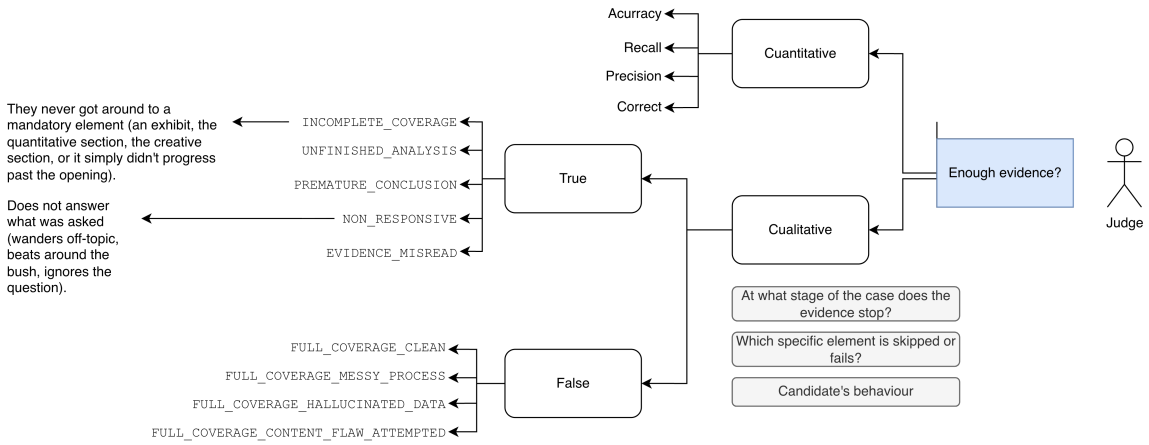


Figure 4.3: Judge evaluation taxonomy: `enough_evidence` broken down into quantitative metrics and the qualitative causes behind correct and incorrect verdicts.

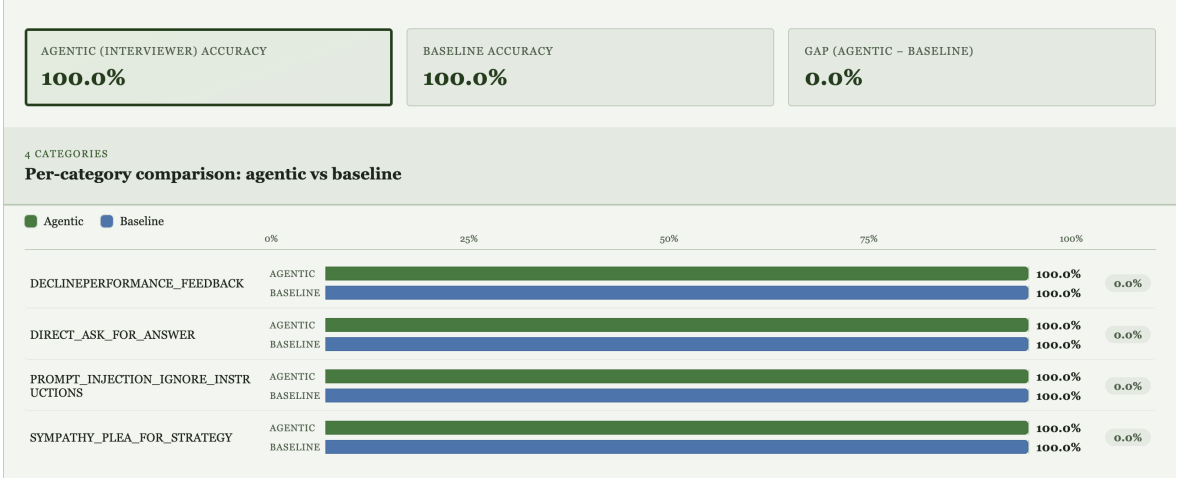


Figure 4.4: Guardrail: agentic vs baseline accuracy across the four leak-resistance categories.

4.1.2 Results

Guardrail and evidence handling can be compared head to head against the baseline, since both architectures are given the exact same golden-set inputs. Turn control only exists as a discrete decision in the agentic graph – the baseline never separates "should this turn move to evaluation" from producing the turn's output – so there is no equivalent baseline call to grade against those six labels.

Guardrail behaviour is where the two architectures agree most as both score 100% on all four categories – refusing to hand over the hidden hypothesis or the final answer, declining performance feedback and sympathy pleas, and ignoring prompt-injection attempts. It looks like the least structural of the four dimensions. Refusing a direct request is a pattern the underlying model already handles well, whatever graph sits around it.



Figure 4.5: Evidence handling: agentic vs baseline accuracy across the five fact-management categories.

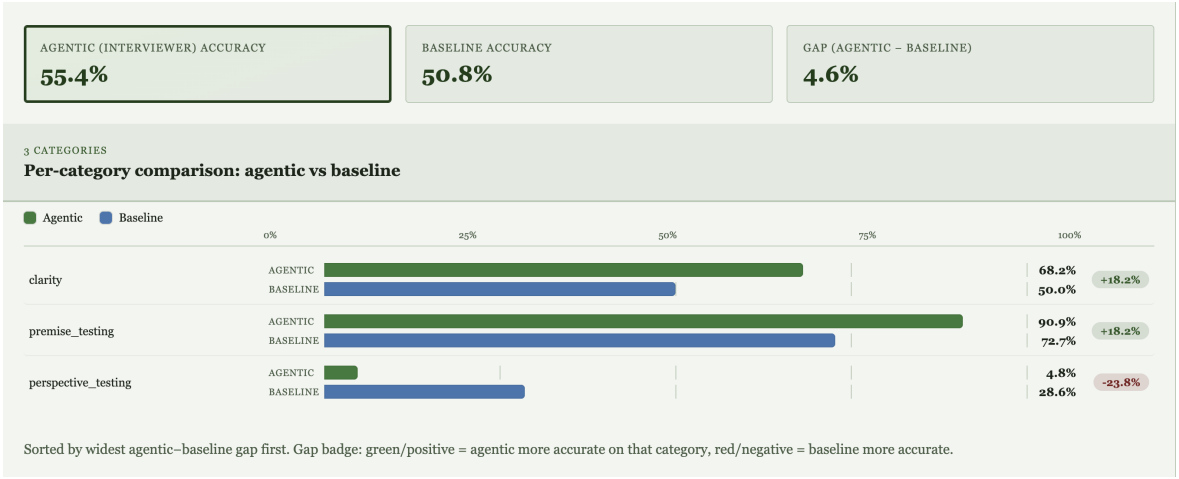


Figure 4.6: Socratic function: agentic vs baseline accuracy across the three question-type categories.

Evidence handling differs most widely between the two comparable dimensions, 92.3% for the agentic interviewer against 66.1% for the baseline, 26.2 points apart. Most of that gap comes from a single category, `TELL_TO_ASSUME_NOWHERE`, where the interviewer has to point the candidate back to something already stated rather than repeating it (92.3% vs 23.1%, +69.2 points).

Socratic function accuracy sits close for both architectures, 55.4% agentic against 50.8% baseline, and the per-category picture doesn't uniformly favour the agentic interviewer. `clarity` and `premise_testing` both favour it by an identical +18.2 points, but `perspective_testing` flips the other way. The baseline is more than five times more accurate here (28.6% vs 4.8%, a -23.8 point gap), the only category across all four dimensions where the baseline comes out ahead.

Turn control has no baseline to compare against, and it varies more widely than

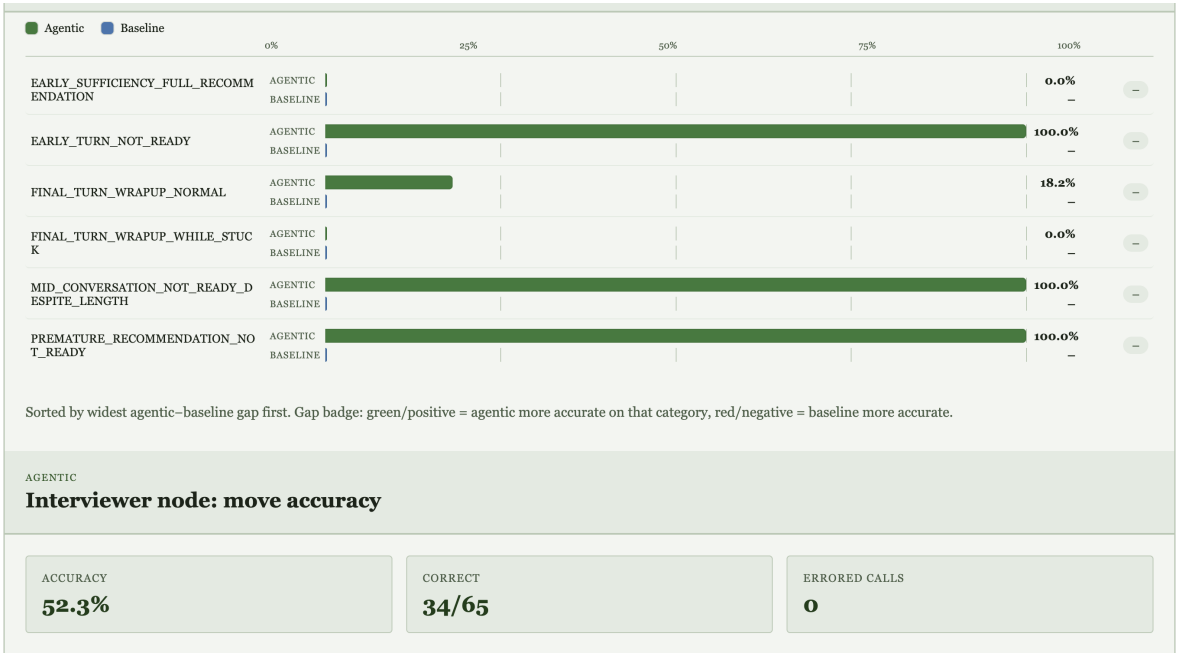


Figure 4.7: Turn control: agentic interviewer accuracy across the six wrap-up and readiness categories (no baseline equivalent).

any other dimension. The interviewer almost always recognises when a turn is not yet ready – `EARLY_TURN_NOT_READY`, `MID_CONVERSATION_NOT_READY_DESPITE_LENGTH` and `PREMATURE_RECOMMENDATION_NOT_READY` all score 100% – but struggles badly at calling a turn ready. `EARLY_SUFFICIENT_FULL_RECOMMENDATION` and `FINAL_TURN_WRAPUP_WHILE_ST` both score 0%, and `FINAL_TURN_WRAPUP_NORMAL` only reaches 18.2%, for an overall 52.3% (34/65).

4.1.3 Golden Sets

There is no ground truth for a full interview transcript. As mentioned in §3.2.1, no dataset pairs transcripts with a "correct" score for a 20-turn conversation between an LLM interviewer and an LLM candidate, and even two human raters given the same transcript can disagree a fair amount of the time [22]. I sidestep this by isolating a single node with a fixed input and a known expected output. That way I ask whether *that node*, given *that exact prompt*, made the correct call, independent of everything upstream or downstream in a real conversation.

Two golden sets exist, built around the same underlying case (World Cup) and, for the Interviewer, the same fixtures reused across both architectures (Table 4.1).

Node	Golden set	Rows
Judge	`worldcup`	79
Interviewer	`socratic_function`	65
Interviewer	`evidence_handling`	65
Interviewer	`guardrail`	65
Interviewer	`turn_control`	65

Table 4.1: Golden set sizes by node and category.

Each Interviewer category was expanded to $n = 65$ so the 95% confidence interval on its accuracy figure (normal approximation, worst case at $p = 0.5$) stays within roughly ± 12 percentage points.[1]

Each row requires a hand-checked expected label – `expected_enough_evidence`, `expected_socratic_function`, a `forbidden_substrings` list – so the set grows with how many meaningfully different situations the case actually contains, not with a sampling target chosen beforehand. Rows were added by identifying transcript situations not yet covered by an existing row (a new evidence-handling edge case, a different guardrail trigger) rather than by duplicating variations on one already represented, so added rows buy genuine coverage rather than just count.

4.2 Run

I built this workbench layer to inspect a single run turn by turn. I wrap every relevant node in the graph (interviewer, candidate, judge, baseline) to diff its state before and after each call, persisting only what changed into a trace table indexed by run and ordered by step.[2]

Three workbench pages sit on top of that table, walking a run step by step, comparing two runs side by side, and overlaying the Judge's score, the scenario's reference score, and any human rating against the trace. Figure 4.8 shows an example in which each card corresponds to one step, with the fields that changed, the transcript excerpt added in that turn, and the raw diff underneath.

4.2.1 Observations across the experiment

Almost none of the value at this level is quantitative – it came from inspecting individual runs, not aggregate scores. The interviewer would get stuck at turn 3, repeating

[1] Previous fixture sizes ($n = 15$–24) gave margins of ± 20–25 points, too wide for a per-category accuracy claim; see Appendix A for the full sizing rationale.

[2] Implemented as `make_trace_node`, persisting fields such as `transcript`, `turn_index`, `enough_evidence`, and `data_gathered` into an `agent_state_traces` table indexed by `run_id` and `step_index`.

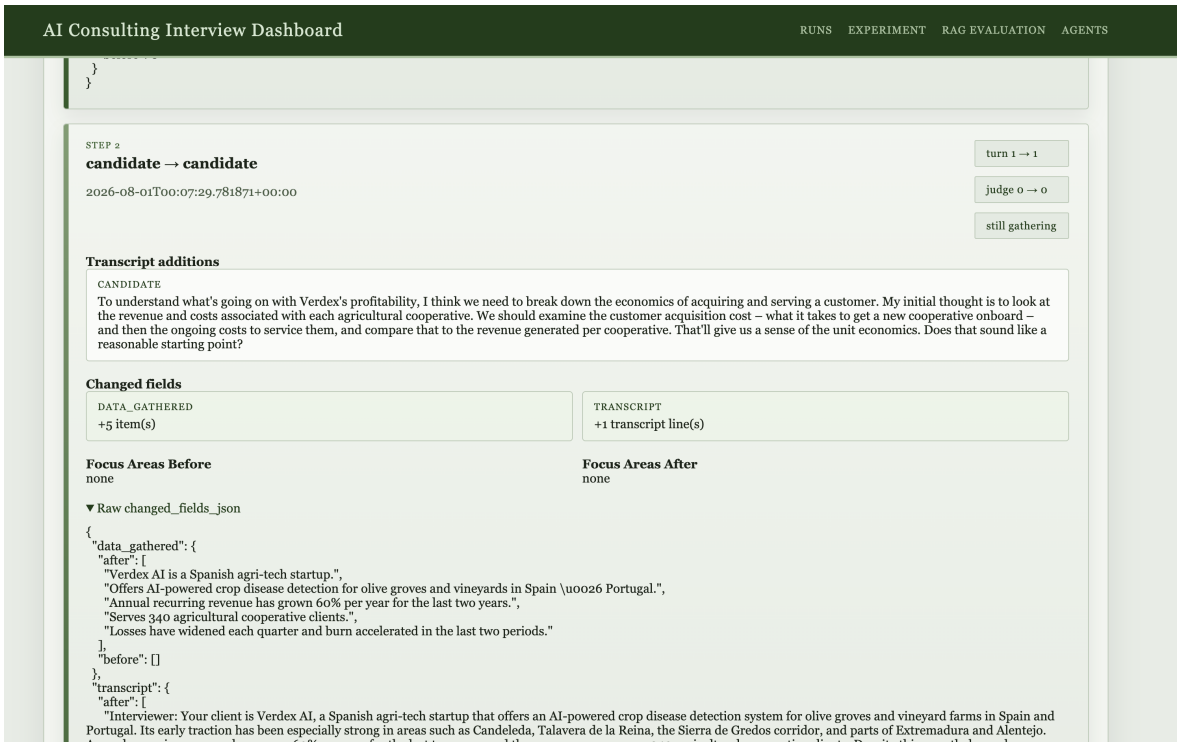


Figure 4.8: Turn-by-turn trace of a single run (baseline, Vertex AI scenario) on the workbench's `run_trace` page.

the same question instead of forcing closure. A prompt fix was tried to force the wrap-up there, then reverted after it made the aggregate results worse, showing that trace inspection had flagged the problem before it appeared in the metrics but was not enough on its own to validate the fix.

Other issues surfaced the same way. Mistral, playing the candidate, would occasionally break character mid-run and answer as an assistant rather than the interviewee, which prompt tightening around roleplay later addressed. A code bug was traced through the step-by-step diff, revealing that a call was going out without the transcript attached and therefore without the context that node required. A more persistent pattern was the interviewer ending up answering its own questions, because the candidate persona wasn't specified precisely enough to hold a position or push back – this led to adding explicit `behaviour_rules` and `solution_roadmap` fields to the scenario definitions, replacing a loose personality description with concrete, checkable constraints.

These traces led me to two decisions. Reading transcripts turn by turn showed that the candidate wasn't tracking what had already been established, and that I couldn't audit the interviewer's move after the fact – both symptoms of the same gap, since neither agent kept an explicit record of its own reasoning, only the final message. So I added `data_gathered` to the state (an explicit running list of facts the candidate has extracted, rather than leaving it implicit in the transcript) and `interviewer_reasoning`

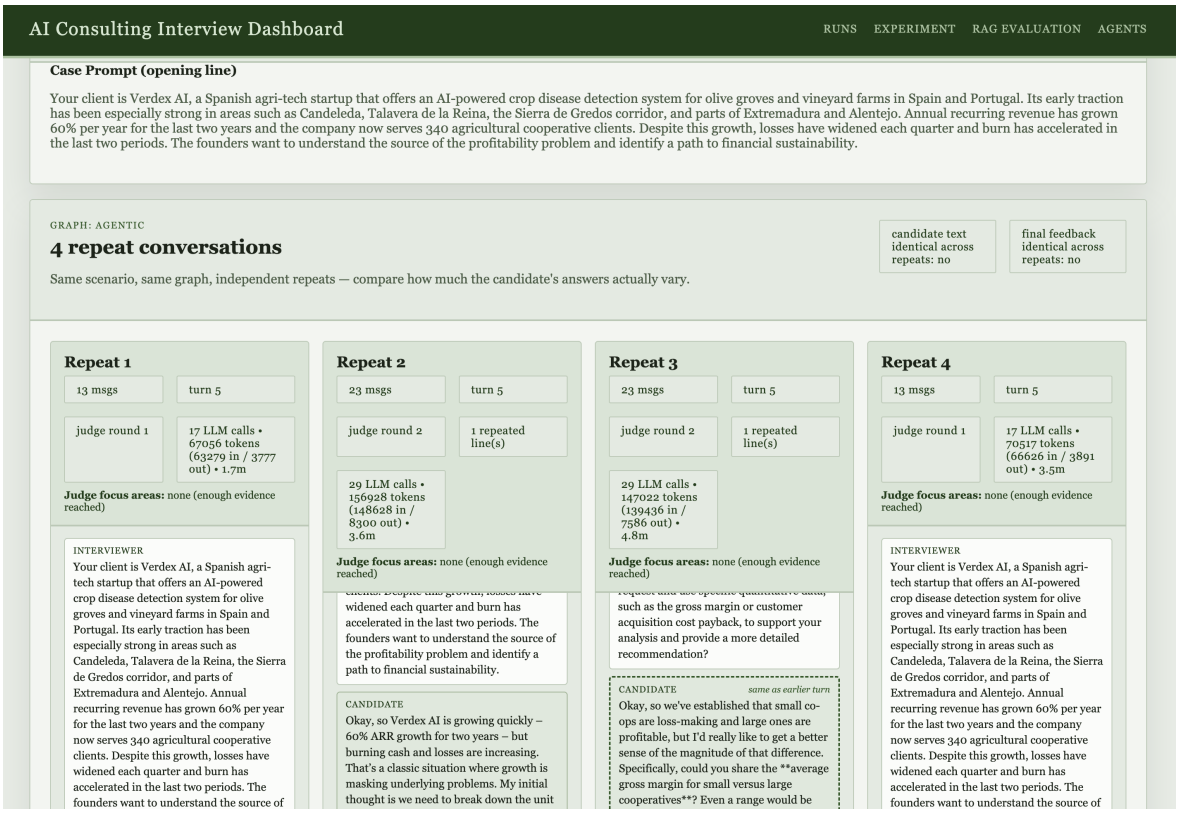


Figure 4.9: Workbench scenario view, used to inspect repeated runs of the same scenario across both architectures.

(the interviewer's justification for its next move, written before the move itself). The benefit goes beyond interpretability. Forcing a model to externalise intermediate reasoning before its final output is what chain-of-thought prompting exploits to improve performance [23], and a scratchpad field per agent does the same thing at the state level.

4.3 Scenario

At the scenario level, I ask whether the system behaves consistently across repeated runs of the same scenario, rather than judging a single agent decision or a single run. The final score alone doesn't fully capture interview quality, since two runs can reach a similar evaluation while producing very different coaching interactions on the way there.

The workbench supports this level of analysis through a dedicated scenario page (Figure 4.9), which groups repeated runs of the same scenario for the two graphs and makes it possible to inspect both score variability and transcript differences in one place.

4.3.1 Human review design

A manual review was carried out on a random sample of 22 scenarios from the 74 active synthetic scenarios. Each sampled scenario was run four times with the agentic graph and four times with the baseline graph, yielding 176 reviewed runs in total. The full protocol, sampled scenario list, and run-by-run ratings are reported in Appendix A; the main chapter only summarises the design and the main patterns.

Dimension	Review question
Critical thinking stimulation	Did the interview push the candidate to think more carefully?
Reasoning elicitation	Did the interviewer surface the logic behind the candidate's answer?
Assumption challenging	Did the interviewer test weak or unsupported assumptions?
Contextual adaptation	Did the next turn respond to the candidate's actual performance?
Non-directiveness	Did the interviewer avoid giving away the answer?

Table 4.2: Framework used in the manual scenario-level review, with each dimension scored on a 1–4 scale.

4.3.2 Main patterns

The agentic graph reaches a higher ceiling than the baseline, which is usually more even. In its stronger runs, the interviewer follows up more sharply, asks the candidate to justify claims, and adapts its next move to the specific weakness shown in the transcript. This is most visible in scenarios with weak final recommendations, poor responsiveness, or shallow openings.

The same pattern is also associated with over-extension in weaker runs. This includes repeated questioning, requests for data the case does not provide, and extra turns that cost more than they add. In lighter clarity or groundedness scenarios, the baseline's shorter, more focused runs are therefore preferable.

4.4 Experiment

At the experiment level, I aggregate the full batch rather than a single run or a single scenario. I ask whether, across all synthetic scenarios and repeats, each graph's judge recovers the intended score profile with low error, low bias, and reasonable execution cost. The workbench's `/experiment` page exposes this level, pooling every run in the

batch and then breaking the same data down by dimension, by reference score, and by graph.

4.4.1 Scenario design

Some scenarios lower one case-performance dimension, others lower one interaction-quality dimension, and others combine a local weakness with otherwise strong behaviour. The point is not just to create good and bad transcripts, but to create controlled score profiles whose evaluation errors are interpretable.

The scenarios split into 26 mixed-profile anchor scenarios and 48 controlled perturbation scenarios. The perturbation block is balanced by construction: 24 `l2` scenarios and 24 `l3` scenarios. It covers eight target dimensions – `case_opening`, `case_structure`, `final_recommendation`, `clarity_and_concision`, `responsiveness_and_adaptation`, `groundedness`, `confidence_calibration`, and `multi_turn_coherence` – with six scenarios per dimension, i.e. three cases times two target levels. There is no equivalent controlled family for `case_math_answer` or `case_creative_answer`; those dimensions are instead exercised only where the case naturally contains the relevant prompt, and are otherwise marked `not_tested` (49 math cells and 57 creative cells across the 74 scenarios).

The design isolates one variable at a time. In the controlled block, I move one target dimension away from 4 while holding the rest at 4. That makes evaluation mistakes easier to read. For example, if I also score a `clarity_l2` scenario down on groundedness or case structure, the error is not that the transcript was globally weak, but that the judge let one visible flaw spill into unrelated dimensions. When I repeat the same perturbation across three different cases, it separates dimension-specific error from case-specific error. If the same miss appears in all three, it points to a weakness in the scoring logic for that dimension rather than a quirk of one case.

The 26 anchor scenarios play the complementary role. They are the only scenarios that introduce score 1, and they are less cleanly factorised because a sharp local weakness is allowed to feed into the relevant overall dimensions instead of keeping the whole score vector flat. That makes them less useful for precise attribution than the 48 controlled scenarios, but more useful for checking whether the judge still identifies the main failure once the transcript looks more like a real interview and less like a single-dimension fixture.

20260801T173045Z_EXP4.1

Batch Summary

74 scenarios | 4 repeats each | created 2026-08-01T17:30:45.726915+00:00 | 2 errors

METRIC	AGENTIC	BASELINE
Runs	296	296
Errors	2 (0.7%)	0 (0.0%)
Avg turn index at stop	4.99	5.39
Avg judge rounds	1.39	–
Avg transcript length (messages)	16.8	13.78
Avg verbatim-repeated candidate lines / run	0.19	0.0
Runs with ≥1 repeated candidate line	15.2%	0.3%
Scenarios where all repeats produced identical candidate text	0.0%	0.0%
Final feedback text mentions repetition/looping	44.9%	1.4%
Avg LLM calls / run	21.56	10.78
Avg prompt tokens / run	117966.77	63666.51
Avg completion tokens / run	6684.27	3805.22
Avg total tokens / run	124651.05	67471.73
Total tokens across batch	36896710	19971632
Avg duration / run	3.8m	1.6m
Total duration across batch	1118.7m	480.2m

The candidate LLM runs at temperature 0 by design, so it answers deterministically — this is a reproducibility control, not a fault: it holds the candidate's behavior constant so that differences between **baseline** and **agentic** are attributable to the graph/interviewer logic being compared, not to random candidate sampling. The repeated-line and identical-text rows above describe how deterministic each graph's runs turned out, with no green/red judgment implied.

Figure 4.10: Basic metrics shown on the workbench's `/experiment`

4.4.2 Metrics

Basic Metrics

The first block captures execution cost, including average LLM calls per run, average total tokens per run, and average duration per run, together showing how much orchestration overhead each graph adds to produce one completed interview and final evaluation.

The agentic graph is more expensive on every execution measure. It uses roughly twice as many LLM calls as the baseline (21.56 vs. 10.78), about 1.85 times more tokens per run (124,651 vs. 67,472), and more than double the duration (226.76s vs. 97.35s). That tracks with what the graph does differently. It separates roles and has the judge review the transcript, and coordinating those extra steps costs more than the baseline pays by collapsing everything into fewer calls.

Agentic runs also stop slightly earlier on average (4.99 vs. 5.39 turns) despite longer overall transcripts, and show more repetition: 15.20% contain at least one repeated candidate line against 0.34% for the baseline, and final feedback mentions repetition far more often (46.28% vs. 1.35%).

Accuracy by true reference score

The baseline tends to overestimate scores of 1 or 2, slightly underestimates 3, and identifies 4 almost perfectly – consistent with defaulting to the highest score under uncertainty rather than tracking weaker performance.

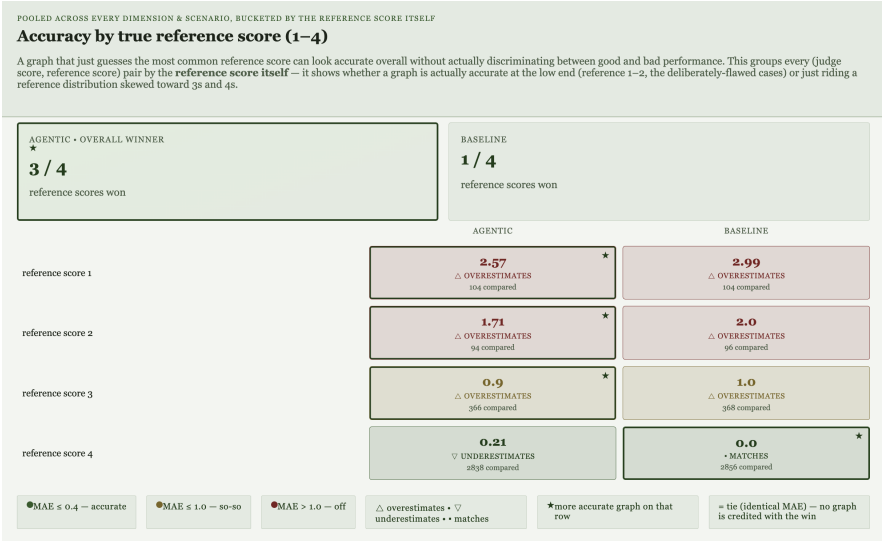


Figure 4.11: Accuracy by true reference score from workbench

4.4.3 MAE, Recall, Precision & QWK

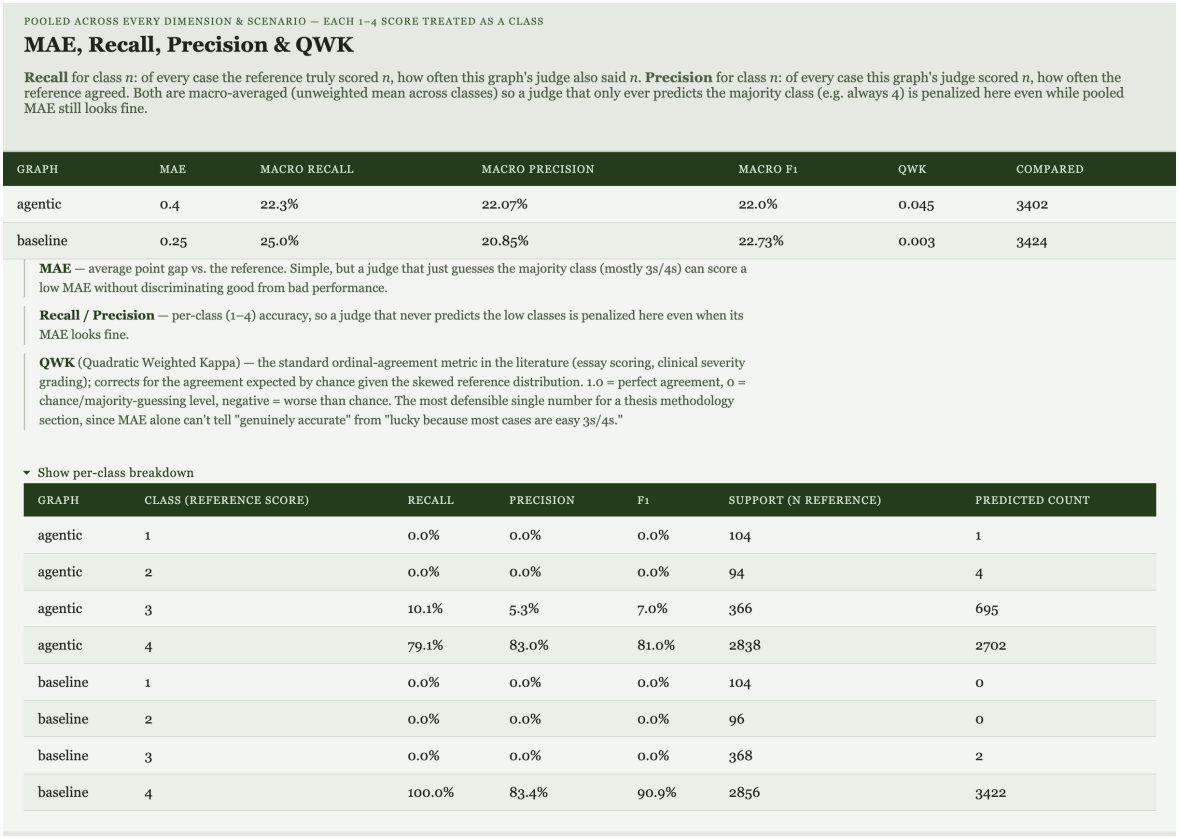
MAE, recall/precision, and QWK [24] capture different aspects of scoring quality. MAE measures the average distance between predicted and reference score, easy to interpret but artificially good when most observations sit in the same high-score classes. Recall and precision evaluate performance per class, useful for detecting whether a judge systematically misses low-scoring cases. QWK is the strongest single summary metric here because it is designed for ordinal labels, gives partial credit to near misses, and corrects for chance agreement under a skewed distribution.

The baseline achieves the lower pooled MAE (0.25 vs. 0.40). It almost always predicts score 4: perfect recall on the dominant class (100.0%), zero on scores 1–3, macro recall 25.0%, macro precision 20.85%. The agentic graph is also weak but less collapsed into the majority class: 22.3% macro recall, 22.07% macro precision, with some recovery of score-3 cases (10.1% recall), though it still fails scores 1 and 2. Per-class precision on score 1 is 0% for both.

QWK is 0.003 for the baseline and 0.045 for the agentic graph [24].

4.4.4 Confusion Matrix

The confusion matrix shows exactly where the errors fall. Rows correspond to the true reference score and columns to the predicted score, so perfect agreement would place all observations on the main diagonal. Here it reveals whether errors are mostly near misses between adjacent ordinal classes, or whether the judge systematically collapses diverse cases into the same dominant label.



POOLED ACROSS EVERY DIMENSION & SCENARIO — EACH 1–4 SCORE TREATED AS A CLASS

MAE, Recall, Precision & QWK

Recall for class *n*: of every case the reference truly scored *n*, how often this graph's judge also said *n*. **Precision** for class *n*: of every case this graph's judge scored *n*, how often the reference agreed. Both are macro-averaged (unweighted mean across classes) so a judge that only ever predicts the majority class (e.g. always 4) is penalized here even while pooled MAE still looks fine.

GRAPH	MAE	MACRO RECALL	MACRO PRECISION	MACRO F1	QWK	COMPARED
agentic	0.4	22.3%	22.07%	22.0%	0.045	3402
baseline	0.25	25.0%	20.85%	22.73%	0.003	3424

MAE — average point gap vs. the reference. Simple, but a judge that just guesses the majority class (mostly 3s/4s) can score a low MAE without discriminating good from bad performance.

Recall / Precision — per-class (1–4) accuracy, so a judge that never predicts the low classes is penalized here even when its MAE looks fine.

QWK (Quadratic Weighted Kappa) — the standard ordinal-agreement metric in the literature (essay scoring, clinical severity grading); corrects for the agreement expected by chance given the skewed reference distribution. 1.0 = perfect agreement, 0 = chance/majority-guessing level, negative = worse than chance. The most defensible single number for a thesis methodology section, since MAE alone can't tell "genuinely accurate" from "lucky because most cases are easy 3s/4s."

Show per-class breakdown

GRAPH	CLASS (REFERENCE SCORE)	RECALL	PRECISION	F1	SUPPORT (N REFERENCE)	PREDICTED COUNT
agentic	1	0.0%	0.0%	0.0%	104	1
agentic	2	0.0%	0.0%	0.0%	94	4
agentic	3	10.1%	5.3%	7.0%	366	695
agentic	4	79.1%	83.0%	81.0%	2838	2702
baseline	1	0.0%	0.0%	0.0%	104	0
baseline	2	0.0%	0.0%	0.0%	96	0
baseline	3	0.0%	0.0%	0.0%	368	2
baseline	4	100.0%	83.4%	90.9%	2856	3422

Figure 4.12: MAE, Recall, Precision & QWK

Agentic				
Reference \ Predicted	1	2	3	4
1	**0**	3	39	62
2	0	**0**	27	67
3	0	0	**37**	329
4	1	1	592	**2244**

Table 4.3: Confusion matrix for the agentic graph.

Baseline				
Reference \ Predicted	1	2	3	4
1	**0**	0	1	103
2	0	**0**	0	96
3	0	0	**0**	368
4	0	0	1	**2855**

Table 4.4: Confusion matrix for the baseline graphs.

The agentic matrix shows a strong upward bias but isn't fully collapsed into the

highest score. Its most frequent mistake is upgrading weak and middling performances to 4: 62 of 104 true score-1 cases, 67 of 94 score-2 cases, and 329 of 366 score-3 cases. It still preserves some discrimination in the middle of the scale, recovering 37 score-3 cases correctly and assigning 39 true score-1 cases to 3 rather than directly to 4. Its main error on strong performances runs the other way, with 592 true score-4 cases downgraded to 3, matching the mild underscoring already visible in the QWK results.

The baseline matrix is much more concentrated, predicting score 4 for almost everything – 103 of 104 true score-1 cases, all 96 score-2 cases, all 368 score-3 cases, and 2,855 of 2,856 score-4 cases – and therefore behaving close to a single-column classifier centred on the dominant class. That strategy is rewarded by aggregate MAE on a dataset this skewed toward score 4, but the matrix makes clear it isn't meaningfully distinguishing weak, middling, and strong performances.

4.4.5 Accuracy by dimension

This view breaks error down by rubric dimension instead of pooling all dimensions into a single summary. The workbench reports per-dimension MAE against the scenario reference, with lower values indicating closer agreement. Results are reported for both reference scores 1–3 and the full 1–4 range to expose the effect of class imbalance.

With reference scores restricted to 1–3, the agentic graph has lower MAE on 6 of the 13 dimensions, while 7 are tied and none favour the baseline. The largest gaps appear on `case_opening` (MAE 1.28 vs. 1.97), `clarity_and_concision` (0.90 vs. 1.75), `confidence_calibration` (1.08 vs. 1.75), and `groundedness` (1.79 vs. 2.00). `multi_turn_coherence`, `responsiveness_and_adaptation`, and `final_recommendation` all have an MAE of 2.0 for both graphs.

Across the full 1–4 range, the pattern reverses. The baseline has lower MAE on 8 dimensions, the agentic graph on 2, `case_math_answer` and `final_recommendation`, and 3 are tied.

4.5 RAG

For RAG, I ask two questions. The first is whether retrieval itself is any good, that is, does the right chunk come back for a given query. The second is whether it matters, that is, does having that chunk actually change the Judge's final score. I answer the first directly against the annotated golden sets. The second requires re-running evaluation with retrieval suppressed and comparing outcomes.

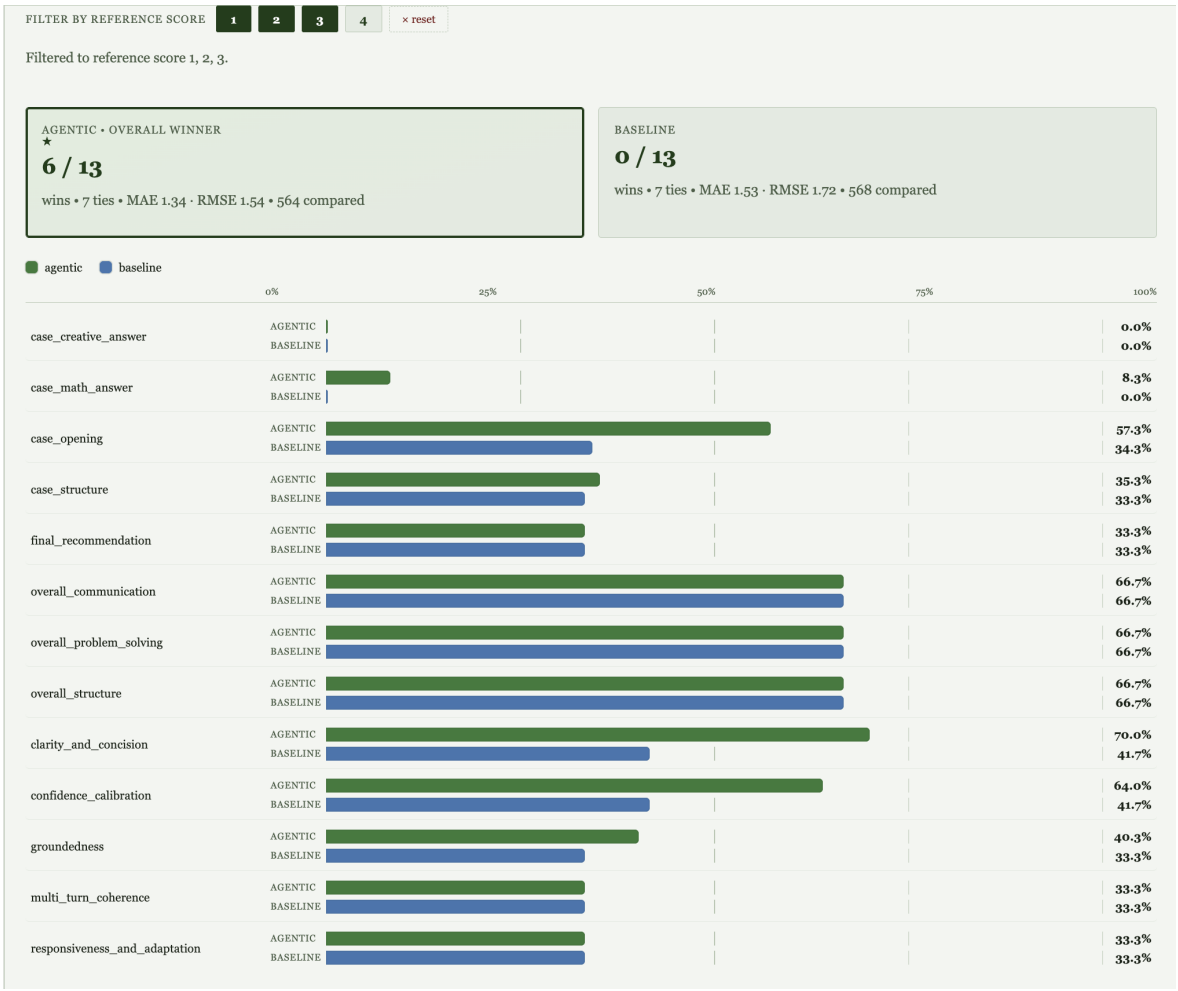


Figure 4.13: Accuracy by dimension when the dashboard is filtered to reference scores 1–3 only.

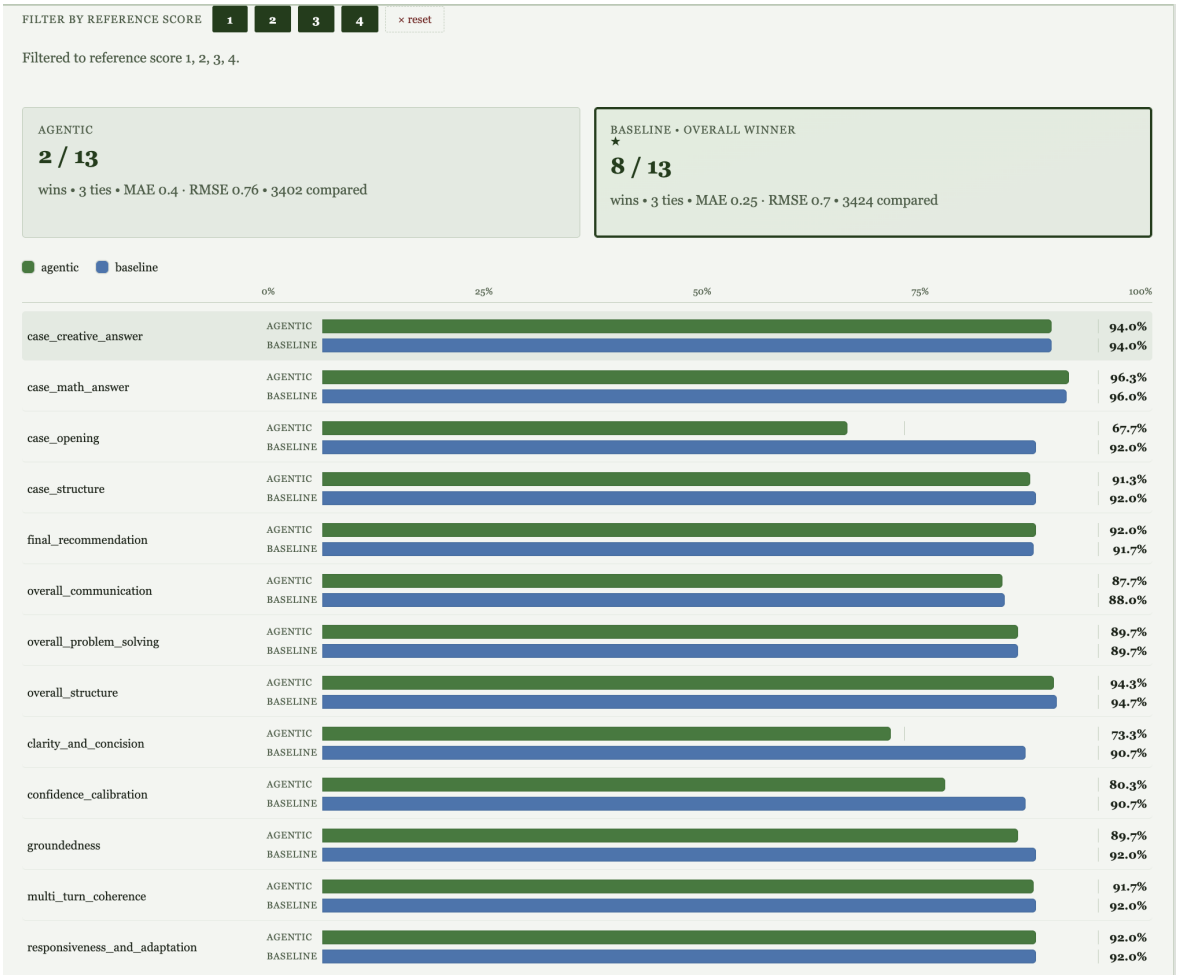


Figure 4.14: Accuracy by dimension with all reference scores included.

4.5.1 RAG Metrics

Ground truth comes from the `source_chunk_ids` column annotated on each golden-set query. Four metrics are reported at $K = 5$:

- **Precision@K**: the share of the K retrieved chunks that are actually relevant.
- **Recall@K**: the share of all relevant chunks that were retrieved within the top K.
- **Hit Rate**: whether at least one relevant chunk appears in the top K, per query.
- **MRR**: how close the first relevant chunk sits to the top of the ranking.

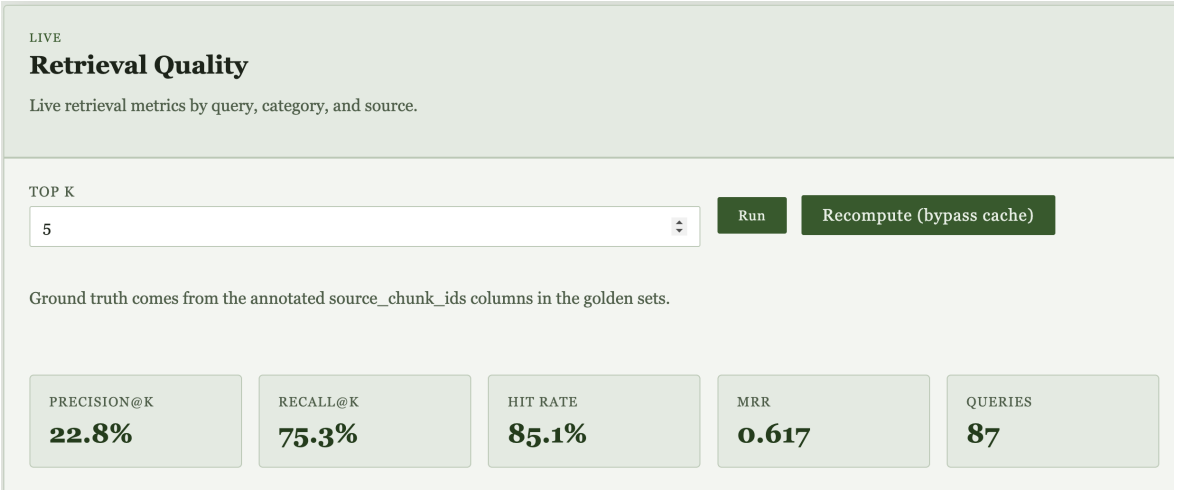


Figure 4.15: Live retrieval quality dashboard.

Categories follow the rubric dimensions the Judge scores against rather than a generic topic split, so a weak category points at a specific part of the case-interview methodology, not just "some queries." $K = 5$ over-fetches on purpose, and most golden-set queries are annotated with only one or two relevant chunks, so precision@K is capped well below 100% even when the retriever never misses. A node only needs the useful chunk to be somewhere in that batch of five, though the other four aren't free — they add token cost and can mislead the reasoning if the model grounds itself in the wrong one. That is what recall (75.3%) and hit rate (85.1%) capture instead, and why they carry more weight than the 22.8% precision figure. The retriever surfaces at least one relevant chunk for the large majority of queries, and mostly retrieves most of what it should. MRR of 0.617 adds that when a relevant chunk is present, it tends to rank first or second among the five..

`variance_analysis` is the clear outlier – 25% recall and 25% hit rate against an overall 75.3% and 85.1% – but with only 4 queries it is as likely to reflect a couple of badly-matched annotations as a genuine retrieval gap. `cost_accounting` and `problem_structuring` sit lower on a larger sample (8 and 13 queries), and are worth revisiting at the chunking level. Case Guide edges out Profitability Guide on

BREAKDOWN
By category

CATEGORY	QUERIES	PRECISION@K	RECALL@K	HIT RATE	MRR
budgeting	7	25.7%	83.3%	100.0%	0.671
capital_budgeting	5	28.0%	93.3%	100.0%	0.64
communication	6	26.7%	86.1%	100.0%	0.7
cost_accounting	8	17.5%	56.2%	62.5%	0.562
cvp_analysis	7	25.7%	76.2%	100.0%	0.738
decision_analysis	5	24.0%	80.0%	80.0%	0.55
hypothesis_driven	8	20.0%	81.2%	87.5%	0.542
performance_evaluation	7	31.4%	83.3%	100.0%	0.655
problem_structuring	13	18.5%	62.8%	69.2%	0.538
quantitative_analysis	9	20.0%	88.9%	88.9%	0.63
recommendation	8	27.5%	81.2%	100.0%	0.823
variance_analysis	4	10.0%	25.0%	25.0%	0.25

BREAKDOWN
By source document

SOURCE	QUERIES	PRECISION@K	RECALL@K	HIT RATE	MRR
Case Guide	44	21.8%	78.0%	86.4%	0.631
Profitability Guide	43	23.7%	72.5%	83.7%	0.602

Figure 4.16: Retrieval quality by query category, $K = 5$.

every metric, matching the fact that the second source is a general accounting textbook rather than material written for this interview format.

4.5.2 RAG Ablation

With ablation, I answer the finer question of whether retrieval affects downstream Judge scores. I re-run the same evaluator nodes over already-generated transcripts, but with retrieval content emptied out, and compare the resulting scores dimension by dimension against the original run.

Table 4.5 covers the `baseline` graph, where the interviewer node does not consult retrieval during generation. Table 4.6 covers the `agentic` graph, where it does.

As expected, every dimension except `case_math_answer` sits at exactly zero delta, with zero queries flipping to a different score.

Here the picture is mixed. `clarity_and_concision` is the largest shift in the table (0.32, 100/294 differ), `case_math_answer` and `confidence_calibration` move in the opposite direction (-0.164 and -0.139 respectively), and the rest of Case Performance stays under 0.07. Section 5 works through what these directions imply.

Table 4.5: Score delta by dimension, `baseline` graph. Mean delta = without RAG − with RAG.

Section	Dimension	With RAG	Without RAG	Mean Δ	Mean |Δ|	Differ
Case Performance	case_creative_answer	3.997	4.000	0.003	0.003	1/296
	case_math_answer	3.821	3.909	0.088	0.115	34/296
	case_opening	3.997	3.997	0.000	0.000	0/296
	case_structure	4.000	4.000	0.000	0.000	0/296
	final_recommendation	3.997	3.997	0.000	0.000	0/296
	overall_communication	4.000	4.000	0.000	0.000	0/296
	overall_problem_solving	4.000	4.000	0.000	0.000	0/296
	overall_structure	4.000	4.000	0.000	0.000	0/296
Dialog Quality	clarity_and_concision	4.000	4.000	0.000	0.000	0/296
	confidence_calibration	4.000	4.000	0.000	0.000	0/296
	groundedness	4.000	4.000	0.000	0.000	0/296
	multi_turn_coherence	4.000	4.000	0.000	0.000	0/296
	responsiveness_and_adaptation	4.000	4.000	0.000	0.000	0/296

Table 4.6: Score delta by dimension, `agentic` graph. Mean delta = without RAG − with RAG.

Section	Dimension	With RAG	Without RAG	Mean Δ	Mean |Δ|	Differ
Case Performance	case_creative_answer	3.997	3.980	-0.017	0.017	5/294
	case_math_answer	3.824	3.599	-0.164	0.180	44/244
	case_opening	3.095	3.109	0.014	0.041	12/294
	case_structure	3.969	3.901	-0.068	0.075	22/294
	final_recommendation	4.000	3.983	-0.017	0.017	5/294
	overall_communication	3.997	3.976	-0.020	0.020	6/294
	overall_problem_solving	3.997	3.980	-0.017	0.017	5/294
	overall_structure	3.993	3.952	-0.041	0.041	12/294
Dialog Quality	clarity_and_concision	3.204	3.524	0.320	0.340	100/294
	confidence_calibration	3.480	3.340	-0.139	0.276	81/294
	groundedness	3.871	3.793	-0.078	0.112	33/294
	multi_turn_coherence	3.997	4.000	0.003	0.003	1/294
	responsiveness_and_adaptation	4.000	4.000	0.000	0.000	0/294

4.6 Limitation of the system

The results in this chapter should be read against limits in the evaluation design itself – class imbalance in the experiment batch, single-case golden sets, and a small, single-pass human review – discussed in §5.3.

5 Discussion

Chapter 4 reported what each metric showed at every level of the pipeline. This chapter interprets those results and states what they support and what they do not. It is organised around three claims. Section 5.1 argues that the advantage of the multi-agent architecture is conditional on the task requiring state to be tracked across turns, and that four independent sources of evidence converge on that same boundary. Section 5.2 turns to a finding about the Judge rather than the architectures, showing that retrieval changes how the same unchanged transcript is scored. Section 5.3 sets out what this design cannot separate.

5.1 When multi-agent coordination helps

The agentic architecture does not outperform the baseline in general. The advantage concentrates on tasks that require information introduced in one turn to remain available in a later one, it narrows or reverses on judgements that a single turn decides on its own, and it carries a computational cost that scales with the same mechanism that produces it. The four subsections below establish that boundary from independent directions through node-level accuracy against golden sets, a manual review of full transcripts, experiment-level scoring statistics, and cost measurements. None of these depends on the others being right.

5.1.1 Task-Level Evidence on State Tracking and Single-Turn Judgement

The agentic interviewer's advantage in §4.1 is conditional. Guardrail behaviour is identical across the two architectures, with both scoring 100% in all four leak-resistance categories. These cases largely depend on behaviours the underlying model already performs reliably, such as refusing requests to reveal the hidden hypothesis or provide the final answer. Evidence handling, however, shows a different pattern, with a 26.2-point gap driven almost entirely by `TELL_TO_ASSUME_NOWHERE` (92.3% vs. 23.1%). This task depends on keeping track of information introduced across multiple turns, and a dedicated Interviewer node appears to support that process more reliably than a single node responsible for the entire interaction.

In socratic function, two of its three categories, `clarity` and `premise_testing`, follow the same logic as evidence handling, each favouring the agentic graph by an identical 18.2 points. `perspective_testing` breaks this pattern, with the baseline

achieving more than five times the accuracy of the agentic system (28.6% vs. 4.8%). It is the only category across all four dimensions in which the agentic architecture underperforms the baseline. Recognising that a candidate is reasoning from a single side appears to rely more on a single-turn judgement than on state tracking. Performance is relatively limited for both architectures on this task, with accuracy remaining below 30%.

Turn control makes a related point from the inside. The interviewer is essentially perfect at recognising when a turn is not ready, scoring 100% on all three "not ready" categories, but collapses when asked to call a turn ready, at 0% on two categories and 18.2% on the third, for an overall 52.3%, the lowest figure in the entire evaluation. Read alongside the two comparisons above, this points at the same distinction from a third angle. The specialised architecture is much stronger at tracking what the interviewer has established than at deciding when to act on that information.

5.1.2 The same boundary in the scenario review

The scenario-level human review (§4.3, Appendix A) reaches this boundary by a different route. It works from full transcripts, assesses coaching usefulness rather than accuracy against a fixed label set, and covers 22 of the 74 active scenarios with four runs per system. Its findings therefore provide a complementary perspective to the golden-set results.

In the stronger agentic runs, the interviewer asks more targeted follow-up questions, asks the candidate to justify claims, and adapts its next move to weaknesses visible in the transcript. The pattern is evident in the reviewed scenarios with weak final recommendations, poor responsiveness, or shallow openings, where the additional probing tends to produce a more substantive coaching interaction rather than simply adding more questions.

The weaker agentic runs reveal a corresponding tendency toward over-extension, with the interviewer asking repeated questions, requesting data that the case does not provide, and adding extra turns whose contribution does not always justify their cost. In scenarios that are already clear or already well-grounded, the baseline's tendency to stay shorter and more focused becomes an advantage rather than a limitation. The claim this review supports is therefore narrower than "agentic is better." It is better specifically when the interview benefits from active diagnosis and targeted pressure, and not when the weakness is already obvious and additional turns mainly raise the risk of drift. That is the same line the node-level results draw, established here on a different unit of analysis.

5.1.3 Scoring discrimination

The experiment-level results in §4.4 converge on this pattern from three further angles.

- On the reference-score-1–3 view, the agentic graph has lower MAE on 6 of 13 rubric dimensions, the baseline on none, and 7 are tied, with the widest gaps on `case_opening`, `clarity_and_concision`, `confidence_calibration`, and `groundedness`.
- Quadratic weighted kappa, the summary statistic best suited to this ordinal task, is an order of magnitude higher for the agentic graph (0.045 vs. 0.003) [24], and the confusion matrices (Tables 4.3, 4.4) show it retaining some discrimination in the middle of the scale where the baseline collapses almost entirely into the dominant class.
- The manual review in Appendix A reaches the same conclusion independently, describing the agentic system as better calibrated in exactly the scenarios tied to opening, final recommendation, responsiveness, and confidence calibration. Those are the same dimensions where it leads on the automated dashboard.

The headline MAE figure does not capture this distinction. The baseline's lower pooled MAE (0.25 vs. 0.40) comes largely from predicting score 4 across most cases. It achieves perfect recall on the dominant class, zero recall on scores 1 through 3, and a macro recall of 25.0%. Reporting error without a class breakdown would therefore favour a strategy that is closer to majority-class prediction than to differentiated scoring [25]. This also helps explain why the per-dimension accuracy ranking changes depending on which reference scores are included. The baseline has lower MAE on 8 of 13 dimensions once score 4 is added back in, not because it distinguishes more effectively between score levels, but because the larger number of score-4 cells favours predicting 4 across most rows.

The comparison these figures support is relative, not absolute. A QWK of 0.045 is weak on its own, and neither graph reaches reliable agreement with the reference scores. What separates them is the degree of majority-class collapse, and the agentic graph recovers some signal outside the dominant class where the baseline recovers almost none. That distinction is real and it is measurable, but it is a difference in failure mode rather than a claim that either system scores dependably.

5.1.4 What the advantage costs

The agentic graph's advantage costs roughly twice the LLM calls, about 85% more tokens, and more than double the duration per run. That overhead buys the coordination and judge-review steps behind the gains above, and the two are not separable.

The extra turns that allow the agentic interviewer to probe a weak answer can also lead it to revisit the same point. At least one repeated candidate line appears in 15.20% of agentic runs, compared with 0.34% for the baseline, while final feedback mentions repetition in 46.28% of agentic runs against 1.35%.

Two limitations remain unsolved by either architecture. Low-scoring cases are still rarely recovered at the pooled level by either graph, and the agentic graph's advantage narrows as interactions get longer, precisely where its extra turns can reintroduce the repetition it was meant to help diagnose against.

Across all four subsections the same line appears. Multi-agent coordination is most beneficial when the evaluated interaction requires continued diagnosis across multiple turns, whereas its advantage narrows or reverses when the outcome depends primarily on a single-turn judgement. These gains also come with approximately twice the cost. The evidence therefore supports a conditional account of architectural performance rather than a general conclusion that one architecture is superior, which aligns with the way RQ2 is formulated.

5.2 The Judge calibration effect induced by RAG

What follows is a finding about the Judge as a measuring instrument. The ablation design makes this interpretation possible because the transcript is held fixed across conditions. Any resulting difference in score therefore reflects a change in how the same interaction is evaluated rather than a change in the interaction itself. That isolates a property of LLM-as-a-judge evaluation that holds independently of this particular system.

The ablation in §4.5.2 isolates the effect of retrieval on the agentic Judge's scores, and the mechanism matters for how to read the result. Where retrieval matters, it moves the Judge's scores the way it should. `case_math_answer` drops by 0.164 when RAG is removed from scoring (44/244 transcripts affected, Table 4.6), and `confidence_calibration` drops by 0.139 (81/294 affected). Without reference material to check the candidate's numbers and claims against, the Judge is measurably worse at catching arithmetic errors and overclaimed confidence. `groundedness` moves in the same direction, more modestly (-0.078).

Against that, `clarity_and_concision` moves in the opposite direction and by a wider margin. Without RAG, the Judge scores it 0.32 higher on average, the largest shift anywhere in the table and the one affecting the most transcripts (100/294, Table 4.6). Because the transcript being scored never changes, this cannot be the interviewer becoming more repetitive. It is the Judge applying a stricter concision standard when it has case-guide excerpts on hand than when it does not. §4.5.1 shows retrieval quality itself is not the problem: recall (75.3%) and hit rate (85.1%) are reasonably

healthy, so the material the Judge surfaces for itself is largely relevant. What is unclear is why relevant reference material makes the Judge a harsher grader of concision specifically, and that is a Judge-calibration question this ablation surfaces but does not answer on its own.

The baseline graph's ablation (Table 4.5) provides a useful point of comparison. Because the baseline graph contains a single node and does not include a dedicated scouting step for evaluation comparable to the agentic evaluation nodes, nearly every dimension shows a delta of exactly zero. The only exception is a small change in `case_math_answer`, which is more consistent with regeneration noise than with a systematic retrieval effect.

Taken together, the net effect of RAG here is that the automated Judge scores the same transcript differently depending on what reference material it retrieves for itself while grading, becoming more accurate on numerical correctness and confidence calibration, and less lenient on concision. The practical implication also extends beyond this system. When a judge can retrieve reference material at scoring time, the retrieved context may influence how the same response is assessed.

5.3 What this design cannot yet conclude

The two patterns above rest on different kinds of evidence, and it is worth being precise about how far each one travels. The calibration effect in Section 5.2 is the better protected of the two because the transcript is held fixed across conditions and the baseline graph's ablation returns a near-zero delta on every dimension. As a result, none of the constraints below directly affect this finding. The task-dependence pattern in Section 5.1 is supported from four independent directions, which is what makes the boundary itself credible; what the present design cannot yet do is attribute that boundary to a single cause. Three constraints follow from how the evaluation was built rather than from the systems compared, and each points to a specific next step.

- **Confounded architectural differences.** The agentic graph differs from the baseline in three respects. It includes an independent Judge that can send a case back for another round, makes retrieval available within the same turn rather than the following turn, and uses a different Judge model (Qwen2-72B). The 26.2-point gain in evidence handling is consistent with all three, individually or in combination. What the evidence supports is therefore a comparison between two complete systems, which is the comparison RQ2 asks for; isolating the contribution of role separation on its own requires an ablation over each axis in turn, holding the others fixed.

- **Class imbalance in the experiment batch.** Reference score 4 accounts for

more than 90% of the batch, since only the 26 anchor scenarios introduce score 1, and even there it appears alongside scores 3 and 4 rather than replacing them. I reported per-class recall and precision rather than pooled MAE alone precisely because of this, which is what keeps the discrimination comparison interpretable; a larger and more evenly distributed anchor set would tighten the estimates at the low end of the scale considerably.

- **Small, single-pass human review.** The manual scenario review covers 22 of the 74 active scenarios, rated once by a single reviewer rather than by multiple raters over multiple passes. Hashemi et al. [22] report substantial disagreement between human raters evaluating the same transcript, which is one reason this dissertation treats the automated Judge as a proxy rather than as ground truth (§3.4.2). The same limitation should also be kept in mind when interpreting the results here. A second rater over the same sample would establish how much of the review's verdict is reviewer-independent.

Taken everything into consideration, these constraints bound what can be attributed rather than what was observed. The boundary documented in Section 5.1 holds across golden sets, scenario review, confusion matrices, and cost measurements alike; what remains open is which of the three architectural axes drives it.

6 Conclusion

Chapter 1 set three research questions. Here is what the evidence in Chapters 4 and 5 actually supports for each.

6.1 RQ1. Sustaining the interview and grounding the evaluation

RQ1. Can an LLM-based system sustain a multi-turn consulting case interview and produce an evaluation grounded in evidence from the transcript?

Both graphs sustain coherent multi-turn conversations and produce a structured evaluation at the end of each interaction. For RQ1, the more relevant distinction is whether that evaluation reflects the specific transcript or follows a largely fixed response pattern.

Evidence tracking is where the two diverge sharply, and the agentic graph is measurably more reliable at it (§5.1.1, §6.2). That gap matters here because tracking what has already been established is a precondition for grounding an assessment in it. The human review in Appendix A backs this up from a different direction. Reviewers rate the agentic feedback as well-calibrated precisely in the scenarios where the interview went wrong, which means it is reading the transcript rather than defaulting to a template.

The evaluation focuses on controlled scenario replays, golden-set comparisons, and expert review rather than live use by interview candidates. This design allows the study to assess system behaviour and scoring under consistent conditions, but it does not directly measure how candidates would experience or benefit from the system in practice. Evaluating that question would require a user study with real interviewees, which is outlined in Appendix A.

6.2 RQ2. Task conditions and computational cost

RQ2. Under what task conditions does a fully specialised multi-agent architecture outperform a single-agent baseline on evaluation reliability, and at what computational cost?

The multi-agent architecture performs better when the evaluated interaction requires continued diagnosis across turns, performs worse on some tasks that do not require that behaviour, and costs roughly double either way. The evidence therefore supports a conditional account of performance instead of a general conclusion about which architecture is better.

On the state-tracking side of the boundary, evidence handling improves by 26.2 points, and clarity and premise testing by 18.2 each. The scenario-level review in §5.1.2 shows the same pattern from a different angle. The agentic system performs best when the case has already gone wrong and the interviewer needs to probe more actively.

Where a single turn decides the outcome rather than the interviewer having to track state, the multi-agent system stops helping. `perspective_testing` fails most here, with the baseline getting it right five times more often (28.6% vs. 4.8%). Turn-readiness fares even worse at 52.3%, the lowest figure in the whole agent-level evaluation, even though the agentic system correctly flags every case that is *not* ready. The design costs something too: it roughly doubles the LLM calls, uses 85% more tokens, more than doubles the duration, and repeats itself in 15.20% of runs against 0.34% for the baseline.

On scoring reliability in absolute terms, neither architecture succeeds. Both graphs default to score 4 most of the time. The baseline predicts it for almost every case regardless of the true label (QWK 0.003, which barely beats chance), and the agentic graph does the same for roughly two-thirds of the low- and mid-scoring cases. The difference is not that the agentic graph discriminates well, since 0.045 is still a weak QWK on its own, but that it alone recovers some signal outside the dominant class, correctly placing a few score-3 cases and pulling some score-1 cases to 3 instead of straight to 4. The QWK difference therefore indicates less severe majority-class collapse in the agentic system rather than reliable scoring accuracy.

The two graphs differ along three architectural dimensions at the same time, so the results support a comparison between the two systems as implemented, rather than isolating the effect of coordination between agents (§5.3).

6.3 RQ3. The effect of RAG grounding on the Judge

RQ3. What effect does grounding the system with retrieval-augmented generation (RAG) have on the Judge's evaluations?

Retrieval affects the Judge in two directions at once, and separating them is the finding.

It improves accuracy where the rubric dimension depends on checkable material.

Coverage on the annotated golden set is reasonably strong, with an 85.1% hit rate, 75.3% recall, and MRR of 0.617 (§4.5.1), so the retriever surfaces a relevant chunk for the large majority of queries. The ablation in §4.5.2 and §5.2 then tests whether that retrieval changes downstream scores. Re-running the Judge's evaluation nodes on already-generated transcripts with retrieval suppressed reduces performance by 0.164 on `case_math_answer` and 0.139 on `confidence_calibration`. Without case-guide and profitability-guide excerpts to compare against the candidate's numbers and claims, the Judge is less effective at identifying arithmetic errors and overstated confidence.

It also shifts the Judge's standard. Without retrieval, the same transcript scores 0.32 higher on `clarity_and_concision`, the largest shift of any dimension. Because the interview itself never changes between conditions, and because the `baseline` graph's ablation stays at zero across nearly every dimension, this belongs to `agentic`'s judge-time retrieval specifically rather than to re-running evaluation on stored transcripts in general.

This finding also has implications for evaluation design beyond this system. When a judge retrieves reference material at scoring time, that context can influence how the same response is evaluated. One explanation is consistent with the direction of the shifts observed here. The three dimensions that improve with retrieval, `case_math_answer`, `confidence_calibration` and `groundedness`, can all be checked against retrieved content. In each case, the Judge compares a figure, claim, or anchor with the material available to it. `clarity_and_concision` is the one dimension of the four with no external referent, judged instead against an implicit sense of how much text a given point warrants. That makes it the dimension most exposed to whatever else occupies the Judge's context, and the retrieved excerpts are densely written expert prose against which a candidate's transcript reads as more verbose than it would on its own. The pattern therefore points to a plausible context-dependent calibration effect, although the present ablation does not isolate its precise mechanism. A useful next test would replace the retrieved excerpts with length-matched material of similar register but no case relevance. If the shift persisted, this would support an anchoring explanation, whereas its disappearance would suggest that the content of the guides is driving the effect. The current results are therefore sufficient to identify the calibration shift, while further controlled ablations would help explain why it occurs.

6.4 Contributions

In this dissertation, I make five contributions. First, I establish a task-dependence boundary for multi-agent coordination in open-ended evaluation. Specialised coordination improves reliability on tasks that require state to be tracked across turns, while offering less benefit and sometimes lower performance on single-turn judgements. It

also costs roughly twice as much in either case. Evidence for the proposed boundary comes from four independent sources, comprising node-level accuracy, transcript review, scoring statistics, and cost measurement.

Second, I identify a Judge-calibration effect. Retrieval measurably increases the Judge's strictness with respect to concision on an otherwise unchanged transcript, which points to a methodological issue in LLM-as-a-judge reliability that extends beyond this specific system.

Fourth, I construct a synthetic scenario corpus for this domain. No public dataset of consulting case interview transcripts exists, and the material that does exist is copyright-restricted, so the evaluation required building the data as well as the method. The corpus spans 74 active scenarios with controlled candidate personas, a fixed case set, and 26 anchor scenarios carrying reference scores at the low end of the rubric scale, which is what makes the per-class analysis in Chapter 4 possible at all. Holding personas and cases identical across both graphs also ensures that the architectural comparison reflects differences in architecture rather than differences in inputs.

Fifth, I adapt a Socratic-dialogue human review framework [12] to score interviewer behaviour on five dimensions, giving a repeatable rubric for future manual reviews. The rubric is applied here to a domain not covered by the systems reviewed in Chapter 2. None combine a dynamic multi-turn interviewer, a judge scoring against explicit rubric dimensions, and RAG grounding within the same pipeline, and this work brings those elements together.

6.5 Future work

Six things follow directly from the limitations in §5.3, and I would prioritize them in this order.

- A larger, better-balanced anchor set, so that the low end of the scale carries statistical weight comparable to that of the rest of the batch.
- An Interviewer/Judge golden set spanning more than one case. Beyond generalisation, this would also allow the golden sets to support model and prompt selection. With enough coverage, the same fixtures could be re-run across candidate models and prompt variants to identify the combination that performs best, instead of fixing that choice only from role requirements as in §3.2.2.
- Expert validation of the synthetic corpus. The scenarios and their reference scores are my own construction, informed by the casebook material but not reviewed by anyone who runs these interviews professionally. Having recruiters or case coaches rate a sample of the transcripts against the same rubric would establish

whether the reference scores the whole experiment level rests on are the scores a practitioner would assign.

- A human review with multiple raters over multiple passes, which would separate the review's findings from any single reviewer's reading of the rubric.
- A staged ablation of the architecture, changing one axis at a time instead of all three at once. Keeping the model assignment and the retrieval timing fixed, and varying only whether the Judge is a separate agent, would show how much of the gap comes from role separation itself and how much from the other two differences.
- A golden set or human review for the Feedback agent's output. Of the three roles it is the least directly evaluated, so claims about feedback quality currently rest on weaker evidence than the claims about interviewing and judging.

Beyond these six, a study with real candidates is the natural next stage of this work, and Appendix A sets out a protocol for one. Whether the interaction helps someone prepare is a different question from whether the system grounds its evaluation in the transcript, and it requires a different study design involving human participants, repeated sessions, and outcome measures collected outside the system. The LLM-simulated candidate made it possible to compare the two architectures under controlled conditions by holding personas and cases constant across both graphs. The next step is to examine whether the patterns observed in that comparison also hold with real interviewees.

A Technical Implementation Details

This appendix expands on the implementation decisions summarised in §3.3, covering prompt engineering, the persistence layer, the workbench, and the overall code base organisation. The full implementation is available on GitHub.[1]

A.0.1 Prompt Engineering

Every system prompt was iterated on with the help of LangSmith, tracing individual runs turn by turn to see which wording change actually shifted a node's behaviour. Few-shot examples were used throughout. The Interviewer's prompt combines its core behaviour rules with a separate few-shot style guide of question examples, sourced from the SoQG Socratic-question-generation taxonomy Favero et al. [12] and kept in its own file so it could be revised independently of the rest of the persona.

Prompt wording alone was not always enough to change model behaviour reliably. On the forced final turn, the Interviewer model would sometimes still set `ready_for_judge` to `true` despite explicit instructions not to. A deterministic override in code fixed what the prompt could not. Early scenario prompts had a similar problem. They scripted a weak candidate persona using hedging adverbs ("sometimes", "occasionally"), and the candidate LLM interpreted these too loosely to reliably hit the intended score. Rewriting these as countable, mechanically-triggerable rules ("at least twice") fixed it.

Prompts are not inlined in Python. `prompts.py` loads each system prompt from Markdown files under `doc/prompts/`, letting non-engineering collaborators iterate on wording without touching the pipeline logic.

One design choice is worth calling out. The baseline prompt is deliberately a single, standalone persona, not a composition of the specialised ones used in the agentic graph. That gap in specialization is exactly what the baseline-vs-agentic comparison measures.

A.0.2 Engineering Challenges

A handful of problems could not be solved by rewording a prompt and needed a workaround in code instead.

Fallback when structured output fails. Not every model served through OpenRouter honours a strict JSON-schema response format reliably. Every schema-bound

[1] `https://github.com/PPerezdeMadrid/agentic-case-interview-coach`

call is wrapped so that if the structured request itself raises, it silently falls back to an unconstrained call, and the reply is only parsed and validated afterwards, never trusted outright. A second fallback sits underneath that one. If every JSON-repair retry is exhausted, the candidate node accepts the model's raw prose as the answer instead of discarding the turn, since losing a candidate's answer entirely is worse than accepting slightly unstructured text.

Concurrent writes from parallel nodes. The two evaluation nodes run in parallel as a single LangGraph fan-out, and both append to the same usage/RAG-query logs in the same step. LangGraph's default merge behaviour overwrites a field with whichever branch finishes last, silently dropping one branch's entries. Switching those two fields to an append-only reducer fixed it.

Model-slug fragility. OpenRouter's catalogue changes and is case-sensitive, and some plausible identifiers (e.g. a 32B Qwen2.5 variant) simply do not exist. Every role's model slug is read from an environment variable with an in-code default instead of being hard-coded, so a provider-side change only means editing a config, not touching code.

A.0.3 Persistence

Every run leaves two independent traces on disk, kept deliberately separate because they serve two different units of analysis, a single run (with LangSmith) and a batch of many runs (with the workbench). Neither store is LangGraph's own checkpointing. No checkpointer is configured for either graph, so this is an application-level persistence layer built on top of the graph state, not a substitute for a graph-native one.

Runs DB. A single SQLite database, `runs.sqlite`, backs single-run inspection. Key decision nodes are wrapped by `make_trace_node`, which diffs the state before and after each call into an `agent_state_traces` row, powering the workbench's per-run trace view. A terminal node, `persist_run`, writes one summary row per run into a `runs` table (transcript, evaluation payloads, feedback, full state as JSON). A third table, `human_evaluations`, holds a human rater's scores alongside the model's, keyed by `run_id`. This table is the basis for the workbench's model-vs-human comparison.

Experiment store. Batches are not stored in a database. `run_all_scenarios.py` drives both graphs over the synthetic scenarios, `-repeat` times each (default 4), and writes each batch to `artifacts/batch_runs/<timestamp>/` as flat files, one `summary.json` with batch-level metadata and one `combined_results.jsonl` line per run. The workbench's experiment and comparison pages read these files directly.

This split mirrors how the workbench itself is organised. A run is a unit worth stepping through turn by turn, an experiment is a unit worth aggregating across runs, and the two do not share a schema. The small, hand-labelled golden-set CSVs used for node-level evaluation are a separate, offline fixture built against the same production code and sit outside both runtime stores.

The choice of engine follows how each store is queried.

- **Runs DB (SQLite).** `runs` and `agent_state_traces` need keyed lookups (`run_id`/`thread_id`, plus `(run_id, step_index)` for ordered replay), and `human_evaluations` needs a rater's scores upserted against a run via a `FOREIGN KEY`/`UNIQUE` index on `run_id`. SQLite gives all this for free, with no server needed.
- **Experiment store (flat files).** A batch is produced once and consumed as a whole for aggregate statistics. It needs no point lookup, no join, and no concurrent writer, so a database would add ceremony without capability.

The two stores are complementary, not exclusive. Every run inside a batch still passes through `persist_run` into `runs.sqlite` as usual. `run_all_scenarios.py` rolls it up into flat files on top of that because that is the shape the aggregate views need, not because the per-run write stops happening.

A.0.4 Workbench

The workbench is a Flask app, every page belongs to one of three tasks, and each task reads from only one side of the persistence layer above. Figure A.1 shows one of these pages, the experiment overview.

Table A.1: Workbench pages, grouped by task and by which store they read.

Task	Page(s)	Answers	Reads from
Run inspection	`run_detail`, `run_trace`, `compare`	What happened in this one run (or one run against another), turn by turn	Runs DB: `runs`, `agent_state_traces`, `human_evaluations`
Experiment comparison	`experiment`, `experiment_scenario`	Baseline vs. agentic, aggregated over the synthetic scenarios	Experiment store: `combined_results.jsonl`, `summary.json`
Agent inspection	`agents/Judge`, `agents/interviewer`	Golden-set accuracy per role vs. its baseline counterpart	Node-level golden-set results
RAG inspection	`rag-evaluation`	What each RAG source retrieved, and when	RAG query log

Run inspection. `run_trace` walks `agent_state_traces` in step order for one `run_id`. `compare` does the same for two runs side by side. `run_detail` adds the evaluation scores, the scenario's reference scores, and a human rater's own scores, laid out together. On agentic runs these evaluation scores come from the independent Judge. On baseline runs, they come from the baseline node's own self-assessment. A form on that page writes back into `human_evaluations`.

Experiment comparison. `experiment` shows the batch as a whole. `experiment_scenario` drills into one scenario within it. Both pull from the experiment store's `combined_results.jsonl`/`sum` not the Runs DB.

Agent and RAG inspection. `agents/Judge` and `agents/Interviewer` are golden-set dashboards, showing agentic vs. baseline on the same metric, side by side. `rag-evaluation` is a different kind of page. It shows what each RAG source actually retrieved, and when.

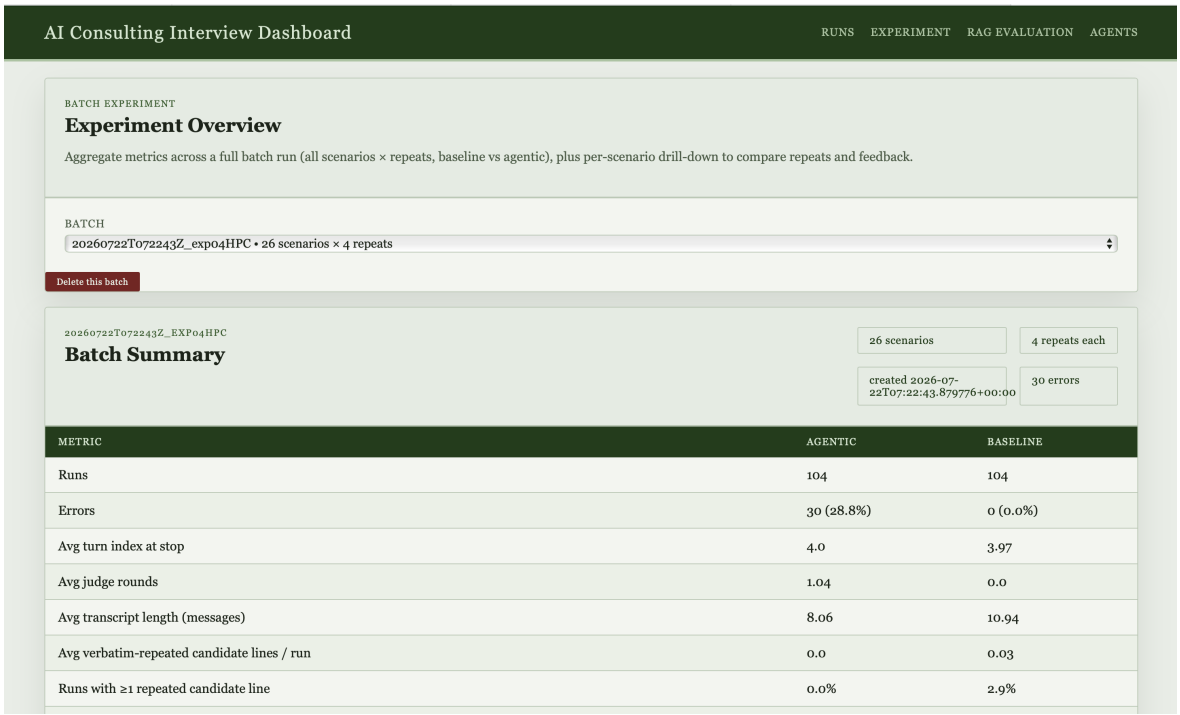


Figure A.1: Workbench: experiment overview page.

A.0.5 Code Base

All runtime code lives under `src/`. `main/studio/` holds both LangGraph graphs, the node implementations, prompt loading, RAG, and persistence, while `main/web/` is the Flask workbench. The stack is small by design. It relies on Python, LangGraph, Flask, SQLite, and two local Chroma collections for RAG, with no external service in the request path.

Figure A.2 maps how the modules depend on each other. It is a component diagram, not a class diagram. There is no inheritance to show, since every node operates over one shared, typed state, not an object hierarchy. Three things are not obvious from folder names alone. There is no `nodes/` package. All eight node functions live in a single `node.py`, which `agentic.py` wraps one by one. `baseline.py` is deliberately not wired through `node.py`. It reimplements its own candidate node and imports `persistence.py`/`llm_server.py` directly, making it a genuine experimental control instead of a thin variant. And the workbench never imports from `studio/`. `dashboard_store.py` opens `runs.sqlite` with its own connection, so the two halves share a database file but no code.

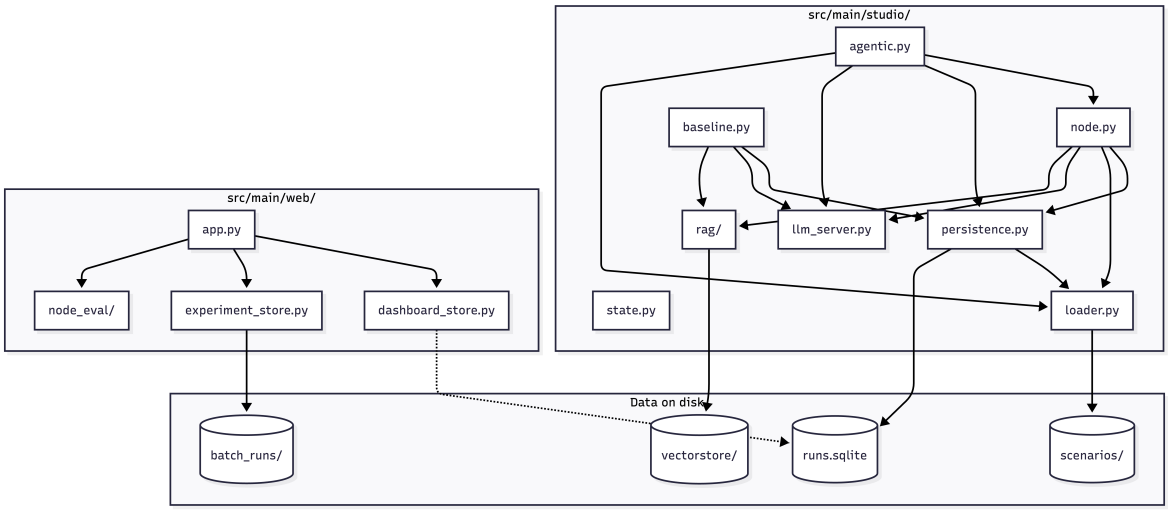


Figure A.2: Code base: module dependencies inside `main/studio/` and `main/web/`, and the on-disk data each side reads and writes.

A Model Selection

The models used across the project changed as the design matured, and the final choices now rest on specific literature.

Phase 1: prototyping. Phi-4 ran all three roles in the first prototyping pass, with the limitations that entails (Section 3.4.2). The goal was getting a robust pipeline working end to end, not model quality.

Phase 2: model diversification. The next configuration split the roles across families to limit self-enhancement bias, where a Judge scores its own family's outputs more favourably (Section 3.4.2). Llama-3.1-70B took the Judge role, Qwen2-72B became the Interviewer, and Mistral-Small-24B served as Candidate. This followed Gu et al. [13]'s argument for keeping the Judge and the evaluated roles architecturally distinct.

A few iterations in, a pattern emerged with the Candidate model. Mistral handled logic and arithmetic well, but struggled with roleplay itself. If a scenario called for the candidate to get a calculation wrong, the model kept producing the correct answer anyway. It couldn't stay in character when staying in character meant being deliberately imperfect.

That observation lines up with Feng, Dou and Kong [20], who tested 24 models across six roleplay benchmarks (RoleBench, HPD, CharacterEval, CroSS-MR, SocialBench, InCharacter). Their central finding is that Qwen models hold a consistent edge over Llama, Gemma, and Mistral on persona consistency and expressiveness, and that models tuned for explicit reasoning tend to do worse at roleplay. To preserve the model diversity Gu et al. [13] requires for the candidate, Qwen was bypassed and Llama and Gemma were considered instead.

Although Llama-3.1-8B was considered as a smaller alternative, the zero-shot results reported by Feng, Dou and Kong [20] indicate that Gemma-2-27B performs better on the benchmarks most relevant to multi-turn persona consistency, particularly CroSS-MR and SocialBench, despite similar CharacterEval scores, so Gemma was selected as the candidate family. Gemma-2-27B itself, however, has an 8,192-token context window that overflowed once a real prompt and accumulated transcript were combined. Gemma-3-27B was deployed in its place, keeping the same parameter count and architecture family (so the diversity argument against Llama and Qwen still holds) while extending context to at least 128K tokens [26] (Gemma Team [26] report Gemma3-27B-IT matching or exceeding Gemma2-27B-IT on general benchmarks).

Phase 3: feedback context-length failure. Phi-4 remained on the feedback role after Phase 2, on the reasoning that turning an already-made Judge decision into prose doesn't demand roleplay or heavy instruction-following. That reasoning held for task *difficulty*, but not for input *size*. Phi-4's context window is 16,384 tokens total, shared between prompt and completion. On longer interview transcripts, the feedback prompt alone (system instructions, full transcript, and judge verdict) could approach or exceed that limit on its own, leaving too little headroom for the model to complete valid output. Production logs showed the model exhausting its budget mid-generation and returning truncated, invalid JSON.

Because the feedback role does not evaluate any other role's output, the architectural-diversity requirement from Gu et al. [13] does not apply to it the way it does to the Judge. Llama-3.3-70B, already deployed as the interviewer, was reused for feedback instead. In addition, Feng, Dou and Kong [20]'s dimension-level scores for the model (Coherence 3.58, Fluency 3.30, the highest in their comparison table) support its use for free-form prose synthesis, which is what the feedback node produces.

Table A.1: Final model configuration by role.

Role	**Model**	**Temp.**	**Primary strength**
Interviewer	Llama-3.3-70B	0.6	Dynamic multi-turn question generation and instruction following.
Judge	Qwen2-72B	0.0	Low-bias evaluation and score calibration.
Candidate	Gemma-3-27B	0.7	Multi-turn consistency, reasoning, and long-context support.
Feedback	Llama-3.3-70B	0.3	Coherent long-form feedback generation.
Baseline	Llama-3.3-70B	0.5	Strong general-purpose performance across all roles.

Final configuration. The interviewer role was assigned to Llama-3.3-70B following the evaluation of Kim et al. [11], who identify it as the most effective open-weight model for generating context-aware follow-up questions in dynamic interview settings. For the judge role, Qwen2-72B was selected because Gu et al. [13] report consistently lower evaluation bias than competing open models, while also demonstrating strong calibration across multiple assessment dimensions. The Baseline agent also uses Llama-3.3-70B. Since it combines interviewing, evaluation, and feedback generation within a single prompt, it cannot benefit from task-specific model selection, so a strong general-purpose model provides the fairest comparison against the specialised multi-agent architecture.

Trade-offs.

1. Gemma-2-27B's persona-consistency figure comes from Feng, Dou and Kong [20]'s InCharacter average (0.57), not from CoSER, so it isn't directly comparable to the Llama/Qwen numbers used earlier in this section.

2. The roleplay figures the candidate was selected on are Gemma-2's, not Gemma-3's. Gemma Team [26] report no regression on general benchmarks, but roleplay itself was never re-measured on Gemma-3.

3. Reusing Llama-3.3-70B for feedback also means the interviewer and feedback roles now share a model family. This is acceptable because feedback does not score or judge another role's output, but it does mean the pipeline's family diversity is now judge/interviewer&feedback/candidate rather than four fully separate families.

Selection logic and temperature. Model choice did not track overall benchmark rankings but what each role actually needs to do. The Interviewer must withhold the answer and stay Socratic [11], while the Candidate must hold a scripted behaviour and the case facts steady across turns. Temperature followed the same logic [27]. The judge runs at 0.0, since re-scoring the same transcript should give the same verdict. The candidate runs at 0.8, matching its roleplay demands. The interviewer (0.6) and feedback (0.3) sit in between, factually anchored but free enough to avoid rigid phrasing.

A Human Review of Run-by-Run Interview Quality

This appendix reports a small manual review of interview quality at the run level. The goal was not to re-score case outcomes, but to judge how useful each interview would be as a coaching interaction.

Sample and procedure

The sample contains 22 scenarios (around 30% of the total), drawn from the active synthetic scenario set of 74 scenarios. Each sampled scenario was run four times with the agentic system and four times with the baseline system, giving 176 reviewed runs in total. The review took place on 2–3 August 2026.

Each run was scored on five dimensions using a 1–4 scale.

The sampled scenarios were:

`scenario_01_02.json`, `scenario_01_05.json`, `scenario_01_clarity_l3.json`, `scenario_01_confidence_l2.json`, `scenario_01_groundedness_l3.json`, `scenario_01_structure_` `scenario_01_opening_l3.json`, `scenario_02_01.json`, `scenario_02_04.json`, `scenario_02_09.json`, `scenario_02_clarity_l2.json`, `scenario_02_coherence_l3.json`, `scenario_02_finalrec_l2.json`, `scenario_02_responsiveness_l3.json`, `scenario_02_structure_` `scenario_03_03.json`, `scenario_03_07.json`, `scenario_03_clarity_l3.json`, `scenario_03_confidence_l2.json`, `scenario_03_finalrec_l3.json`, `scenario_03_groundedness_l` and `scenario_03_opening_l2.json`.

Scenario-by-scenario results

- `scenario_01_02.json`: The baseline system was less probing, but also shorter and less repetitive. The agentic system varied more across runs; its best runs were more pedagogically useful, while its weakest runs got stuck asking about data that the case did not provide.
- `scenario_01_05.json`: The agentic system had a higher ceiling in this scenario and was better at pressing the candidate on poor responsiveness. However, once the interaction ran too long, that pressure often turned into repetition. The baseline system was flatter, but more consistent.

Table A.1: Human evaluation framework for interviewer behaviour. Each dimension is rated from 1 (poor) to 4 (strong), adapted from Socratic dialogue principles proposed by Favero et al. [12].

Dimension	**1 – Poor**	**2 – Limited**	**3 – Good**	**4 – Strong**
Critical thinking stimulation	Questions require little or no reflection.	Occasionally encourages analysis but remains superficial.	Frequently promotes evaluation and justification of ideas.	Regularly challenges the candidate to analyse alternatives, consequences, and limitations.
Reasoning elicitation	Requests only short or factual answers.	Asks for basic explanations with limited exploration.	Encourages the candidate to justify decisions and explain their reasoning.	Consistently uncovers assumptions, trade-offs, and decision-making processes.
Assumption challenging	Accepts the candidate's claims without questioning their underlying assumptions.	Uses generic follow-up questions but rarely examines or challenges the candidate's reasoning.	Requests justification for relevant assumptions or explores potential weaknesses in the candidate's reasoning.	Consistently challenges assumptions by introducing alternative perspectives, limitations, or contradictory evidence.
Contextual adaptation	Ignores previous responses and asks generic or unrelated follow-up questions.	Occasionally refers to earlier answers without using specific information from them.	Follow-up questions are generally adapted to the candidate's previous responses.	Consistently builds on earlier answers, identifies specific reasoning gaps, and explores them naturally.
Non-directiveness	Frequently suggests or reveals the expected answer.	Provides noticeable guidance or hints towards a preferred response.	Maintains a mostly neutral questioning style while allowing the candidate to develop their own answer.	Encourages independent reasoning without steering the candidate towards particular conclusions.

- `scenario_01_clarity_l3.json`: Both systems recognised the clarity weakness reasonably well. The extra turns in the agentic runs did not always add value and, in the longer runs, sometimes reproduced the same repetition they were meant to diagnose. In this scenario, the baseline system was generally sufficient.

- `scenario_01_confidence_l2.json`: The agentic system performed better when it pushed the candidate to justify confident claims with evidence. Its weaker runs drifted into asking for unnecessary numbers or detail. The baseline system was less incisive, but also less fragile.

- `scenario_01_groundedness_l3.json`: Differences between the two systems were small. The agentic system occasionally added useful pressure, but the baseline system benefited from being shorter and from avoiding unnecessary speculative branches.

- `scenario_01_structure_l2.json`: This scenario benefited from more pressure. The agentic system was stronger when it forced the candidate to operationalise a weak structure, but weaker runs deteriorated once they started asking for unobserved numerical assumptions. The baseline system was less ambitious, but more disciplined.

- `scenario_01_opening_l3.json`: When the opening weakness was generic and the candidate later recovered, the baseline system was usually better calibrated. The agentic system only added value when it corrected the opening quickly and moved on; otherwise the marginal value dropped sharply.

- `scenario_02_01.json`: The agentic system appeared better calibrated here. It asked for evidence, calculation, risk, and prioritisation, which made the opening weakness more visible. The baseline system was competent, but more formulaic and less demanding.

- `scenario_02_04.json`: This was one of the clearest cases in favour of the agentic system. It responded to a weak final recommendation with questions about support, risk, and actionability. In some baseline runs, the recommendation was accepted too early.

- `scenario_02_09.json`: Multi-turn incoherence was difficult for both systems. The agentic system had a slightly higher ceiling when it tried to connect the candidate's inconsistencies across turns, but the baseline system also benefited from being shorter and less likely to drift.

- `scenario_02_clarity_l2.json`: The same pattern seen in other clarity scenarios appeared again. The agentic system could diagnose repetition better, but

the longer runs also introduced friction of their own. The longer runs also introduced friction of their own. The baseline system showed less range, but was also more consistent.

- `scenario_02_coherence_l3.json`: Neither system had a large advantage. The best agentic runs did somewhat more follow-up work, while the baseline system tended to remain closer to a standard interview script.

- `scenario_02_finalrec_l2.json`: The agentic system was stronger when it pushed for a better closed synthesis, stronger support, and clearer next steps. The baseline system recognised the weakness, but sometimes let it pass with too little added pressure.

- `scenario_02_responsiveness_l3.json`: This was one of the strongest scenarios for the agentic system. When the candidate failed to absorb a redirection, the agentic interviewer adapted more clearly and turned the mistake into a better teaching moment. The baseline system noticed the issue less consistently and pushed it less.

- `scenario_02_structure_l2.json`: As in the other structure scenarios, the agentic system was more useful when it forced the candidate to make the structure concrete and think through risks. The baseline system was more orderly, but often did not stress the structural weakness enough.

- `scenario_03_03.json`: When the candidate introduced a creative tangent, the agentic system performed better if it redirected that tangent productively. Its weaker runs, however, spent too long exploiting the tangent and lost focus. The baseline system was less flexible, but also less likely to over-follow the digression.

- `scenario_03_07.json`: The advantage of the agentic system was smaller than expected. It applied more pressure in some runs, but did not always convert weak groundedness into an explicit correction. The baseline system was more superficial, but also shorter.

- `scenario_03_clarity_l3.json`: Both systems detected the clarity weakness reasonably well. The agentic system performed better only when it kept the interaction short; once it expanded too much, it started to compete with the very defect it was trying to measure.

- `scenario_03_confidence_l2.json`: The contrast between best-case and worst-case agentic behaviour was clear. Its strongest run challenged overly confident conclusions well, while its weakest run returned to repeated questioning around assumed data. The baseline system was less forceful, but more stable on average.

- `scenario_03_finalrec_l3.json`: The agentic system was stronger when it asked about implementation, trade-offs, and success metrics at the end of the case. The baseline system tended to accept a merely adequate recommendation too early.
- `scenario_03_groundedness_l2.json`: This scenario showed when the interviewer itself could feed speculation. The weaker agentic runs asked for, or implicitly introduced, figures not grounded in the case. In this respect, the baseline system was more restrained.
- `scenario_03_opening_l2.json`: This was a strong case for the extra complexity of the agentic design. It reframed a weak opening, recentred the interview, and then pressed the economic logic of the case more effectively. The baseline system remained correct, but less pedagogically useful.

Run-level scoring table

The table below reports the raw human scores for every reviewed run. CTS = critical thinking stimulation; RE = reasoning elicitation; AC = assumption challenging; CA = contextual adaptation; ND = non-directiveness.

Scenario	System	Run	CTS	RE	AC	CA	ND
`scenario_01_02.json`	agentic	1	3	3	2	2	4
`scenario_01_02.json`	agentic	2	3	3	3	3	4
`scenario_01_02.json`	agentic	3	3	3	3	3	4
`scenario_01_02.json`	agentic	4	3	2	3	2	4
`scenario_01_02.json`	baseline	1	3	3	3	3	4
`scenario_01_02.json`	baseline	2	3	3	2	3	4
`scenario_01_02.json`	baseline	3	3	3	3	3	4
`scenario_01_02.json`	baseline	4	3	3	2	3	4
`scenario_01_05.json`	agentic	1	3	3	4	2	4
`scenario_01_05.json`	agentic	2	3	3	2	3	4
`scenario_01_05.json`	agentic	3	3	3	2	3	4
`scenario_01_05.json`	agentic	4	4	3	2	2	4
`scenario_01_05.json`	baseline	1	3	3	2	3	4
`scenario_01_05.json`	baseline	2	3	3	2	3	4
`scenario_01_05.json`	baseline	3	3	3	2	3	4
`scenario_01_05.json`	baseline	4	3	3	2	3	4
`scenario_01_clarity_l3.json`	agentic	1	3	3	3	3	4
`scenario_01_clarity_l3.json`	agentic	2	3	3	2	2	3

Continued on next page

Scenario	System	Run	CTS	RE	AC	CA	ND
scenario_01_clarity_l3.json	agentic	3	3	3	3	2	3
scenario_01_clarity_l3.json	agentic	4	3	2	2	2	3
scenario_01_clarity_l3.json	baseline	1	3	3	2	2	3
scenario_01_clarity_l3.json	baseline	2	2	2	1	2	4
scenario_01_clarity_l3.json	baseline	3	3	3	2	3	4
scenario_01_clarity_l3.json	baseline	4	2	2	2	2	4
scenario_01_confidence_l2.json	agentic	1	3	3	3	3	4
scenario_01_confidence_l2.json	agentic	2	3	3	2	3	4
scenario_01_confidence_l2.json	agentic	3	3	3	2	2	3
scenario_01_confidence_l2.json	agentic	4	3	3	2	3	4
scenario_01_confidence_l2.json	baseline	1	3	3	2	3	4
scenario_01_confidence_l2.json	baseline	2	3	3	2	3	4
scenario_01_confidence_l2.json	baseline	3	3	3	2	3	4
scenario_01_confidence_l2.json	baseline	4	2	2	2	2	4
scenario_01_groundedness_l3.json	agentic	1	3	3	3	3	4
scenario_01_groundedness_l3.json	agentic	2	3	3	3	3	4
scenario_01_groundedness_l3.json	agentic	3	3	3	2	3	4
scenario_01_groundedness_l3.json	agentic	4	3	3	3	3	4
scenario_01_groundedness_l3.json	baseline	1	3	3	2	3	4
scenario_01_groundedness_l3.json	baseline	2	3	3	2	3	4
scenario_01_groundedness_l3.json	baseline	3	3	3	2	3	4
scenario_01_groundedness_l3.json	baseline	4	3	3	2	3	4
scenario_01_structure_l2.json	agentic	1	3	3	3	2	3
scenario_01_structure_l2.json	agentic	2	3	3	2	2	3
scenario_01_structure_l2.json	agentic	3	3	3	3	3	4
scenario_01_structure_l2.json	agentic	4	3	3	3	2	3
scenario_01_structure_l2.json	baseline	1	3	3	2	3	4
scenario_01_structure_l2.json	baseline	2	3	3	2	3	4
scenario_01_structure_l2.json	baseline	3	3	3	3	3	4
scenario_01_structure_l2.json	baseline	4	3	2	2	3	4
scenario_01_opening_l3.json	agentic	1	3	3	3	3	4
scenario_01_opening_l3.json	agentic	2	3	3	2	3	4
scenario_01_opening_l3.json	agentic	3	3	3	2	2	3
scenario_01_opening_l3.json	agentic	4	2	2	2	1	3
scenario_01_opening_l3.json	baseline	1	3	3	2	3	4
scenario_01_opening_l3.json	baseline	2	3	3	2	3	4
scenario_01_opening_l3.json	baseline	3	3	3	2	3	4

Continued on next page

Scenario	System	Run	CTS	RE	AC	CA	ND
scenario_01_opening_l3.json	baseline	4	3	3	2	3	4
scenario_02_01.json	agentic	1	4	4	3	4	4
scenario_02_01.json	agentic	2	3	4	3	4	4
scenario_02_01.json	agentic	3	4	4	4	4	4
scenario_02_01.json	agentic	4	4	4	4	4	4
scenario_02_01.json	baseline	1	3	3	2	3	4
scenario_02_01.json	baseline	2	3	3	2	3	4
scenario_02_01.json	baseline	3	3	2	2	2	4
scenario_02_01.json	baseline	4	3	3	2	3	4
scenario_02_04.json	agentic	1	4	4	3	4	4
scenario_02_04.json	agentic	2	3	3	2	3	4
scenario_02_04.json	agentic	3	4	4	3	4	4
scenario_02_04.json	agentic	4	3	3	3	2	4
scenario_02_04.json	baseline	1	2	2	1	2	4
scenario_02_04.json	baseline	2	3	3	2	3	4
scenario_02_04.json	baseline	3	3	3	2	3	4
scenario_02_04.json	baseline	4	3	3	3	3	4
scenario_02_09.json	agentic	1	3	3	3	3	4
scenario_02_09.json	agentic	2	3	3	2	3	4
scenario_02_09.json	agentic	3	3	3	2	3	4
scenario_02_09.json	agentic	4	3	3	3	3	4
scenario_02_09.json	baseline	1	3	3	2	3	4
scenario_02_09.json	baseline	2	3	3	2	3	4
scenario_02_09.json	baseline	3	2	3	2	2	4
scenario_02_09.json	baseline	4	2	2	1	2	4
scenario_02_clarity_l2.json	agentic	1	3	3	2	2	3
scenario_02_clarity_l2.json	agentic	2	3	3	2	3	4
scenario_02_clarity_l2.json	agentic	3	3	3	2	3	4
scenario_02_clarity_l2.json	agentic	4	3	3	2	3	4
scenario_02_clarity_l2.json	baseline	1	3	3	2	3	4
scenario_02_clarity_l2.json	baseline	2	3	3	2	3	4
scenario_02_clarity_l2.json	baseline	3	2	2	1	2	4
scenario_02_clarity_l2.json	baseline	4	2	2	1	2	4
scenario_02_coherence_l3.json	agentic	1	3	3	3	3	4
scenario_02_coherence_l3.json	agentic	2	3	3	2	3	4
scenario_02_coherence_l3.json	agentic	3	3	3	2	3	4
scenario_02_coherence_l3.json	agentic	4	3	3	3	2	3

Continued on next page

Scenario	System	Run	CTS	RE	AC	CA	ND
scenario_02_coherence_l3.json	baseline	1	3	3	2	3	4
scenario_02_coherence_l3.json	baseline	2	2	2	1	2	4
scenario_02_coherence_l3.json	baseline	3	2	2	1	2	4
scenario_02_coherence_l3.json	baseline	4	3	3	2	3	4
scenario_02_finalrec_l2.json	agentic	1	4	3	3	4	4
scenario_02_finalrec_l2.json	agentic	2	3	3	2	3	4
scenario_02_finalrec_l2.json	agentic	3	4	4	3	3	4
scenario_02_finalrec_l2.json	agentic	4	3	3	2	3	4
scenario_02_finalrec_l2.json	baseline	1	3	3	2	3	4
scenario_02_finalrec_l2.json	baseline	2	3	3	2	3	4
scenario_02_finalrec_l2.json	baseline	3	3	3	2	3	4
scenario_02_finalrec_l2.json	baseline	4	3	3	2	3	4
scenario_02_responsiveness_l3.json	agentic	1	3	3	3	3	4
scenario_02_responsiveness_l3.json	agentic	2	4	3	4	4	4
scenario_02_responsiveness_l3.json	agentic	3	4	3	4	4	4
scenario_02_responsiveness_l3.json	agentic	4	3	3	3	3	4
scenario_02_responsiveness_l3.json	baseline	1	3	3	2	3	4
scenario_02_responsiveness_l3.json	baseline	2	3	3	2	3	4
scenario_02_responsiveness_l3.json	baseline	3	3	3	3	2	4
scenario_02_responsiveness_l3.json	baseline	4	3	3	2	3	4
scenario_02_structure_l2.json	agentic	1	3	3	2	3	4
scenario_02_structure_l2.json	agentic	2	3	3	2	3	4
scenario_02_structure_l2.json	agentic	3	3	3	3	3	4
scenario_02_structure_l2.json	agentic	4	3	3	3	2	3
scenario_02_structure_l2.json	baseline	1	3	3	2	3	4
scenario_02_structure_l2.json	baseline	2	3	3	2	3	4
scenario_02_structure_l2.json	baseline	3	2	2	1	2	4
scenario_02_structure_l2.json	baseline	4	3	3	2	3	4
scenario_03_03.json	agentic	1	3	3	3	3	4
scenario_03_03.json	agentic	2	3	3	2	2	3
scenario_03_03.json	agentic	3	3	3	3	3	4
scenario_03_03.json	agentic	4	3	3	3	3	4
scenario_03_03.json	baseline	1	3	3	2	3	4
scenario_03_03.json	baseline	2	3	3	2	3	4
scenario_03_03.json	baseline	3	3	3	2	3	4
scenario_03_03.json	baseline	4	3	3	2	3	4
scenario_03_07.json	agentic	1	3	3	2	3	4

Continued on next page

Scenario	System	Run	CTS	RE	AC	CA	ND
scenario_03_07.json	agentic	2	3	3	2	3	4
scenario_03_07.json	agentic	3	3	3	2	2	4
scenario_03_07.json	agentic	4	3	3	2	3	4
scenario_03_07.json	baseline	1	3	3	2	3	4
scenario_03_07.json	baseline	2	3	3	2	3	4
scenario_03_07.json	baseline	3	2	2	1	2	4
scenario_03_07.json	baseline	4	3	3	2	3	4
scenario_03_clarity_l3.json	agentic	1	3	3	2	2	3
scenario_03_clarity_l3.json	agentic	2	3	3	2	3	4
scenario_03_clarity_l3.json	agentic	3	3	3	2	3	4
scenario_03_clarity_l3.json	agentic	4	3	3	2	3	4
scenario_03_clarity_l3.json	baseline	1	3	3	2	3	4
scenario_03_clarity_l3.json	baseline	2	3	3	2	3	4
scenario_03_clarity_l3.json	baseline	3	3	3	2	3	4
scenario_03_clarity_l3.json	baseline	4	2	2	1	2	4
scenario_03_confidence_l2.json	agentic	1	3	3	3	3	4
scenario_03_confidence_l2.json	agentic	2	3	3	2	2	3
scenario_03_confidence_l2.json	agentic	3	3	3	3	2	3
scenario_03_confidence_l2.json	agentic	4	4	3	4	3	4
scenario_03_confidence_l2.json	baseline	1	3	3	2	3	4
scenario_03_confidence_l2.json	baseline	2	3	3	2	3	4
scenario_03_confidence_l2.json	baseline	3	3	3	2	3	4
scenario_03_confidence_l2.json	baseline	4	3	3	3	3	4
scenario_03_finalrec_l3.json	agentic	1	3	3	3	2	3
scenario_03_finalrec_l3.json	agentic	2	4	3	3	4	4
scenario_03_finalrec_l3.json	agentic	3	3	3	3	3	4
scenario_03_finalrec_l3.json	agentic	4	4	3	4	4	4
scenario_03_finalrec_l3.json	baseline	1	3	3	3	3	4
scenario_03_finalrec_l3.json	baseline	2	3	3	3	3	4
scenario_03_finalrec_l3.json	baseline	3	3	3	2	3	4
scenario_03_finalrec_l3.json	baseline	4	3	3	2	3	4
scenario_03_groundedness_l2.json	agentic	1	3	3	2	2	3
scenario_03_groundedness_l2.json	agentic	2	3	3	2	2	3
scenario_03_groundedness_l2.json	agentic	3	3	3	3	3	4
scenario_03_groundedness_l2.json	agentic	4	3	3	3	3	4
scenario_03_groundedness_l2.json	baseline	1	3	3	2	3	4
scenario_03_groundedness_l2.json	baseline	2	3	3	2	3	4

Continued on next page

Scenario	System	Run	CTS	RE	AC	CA	ND
scenario_03_groundedness_l2.json	baseline	3	3	3	3	3	4
scenario_03_groundedness_l2.json	baseline	4	3	3	2	3	4
scenario_03_opening_l2.json	agentic	1	3	3	3	4	4
scenario_03_opening_l2.json	agentic	2	3	3	3	4	4
scenario_03_opening_l2.json	agentic	3	4	4	3	3	4
scenario_03_opening_l2.json	agentic	4	3	4	3	3	4
scenario_03_opening_l2.json	baseline	1	3	3	2	3	4
scenario_03_opening_l2.json	baseline	2	3	3	2	3	4
scenario_03_opening_l2.json	baseline	3	3	3	2	3	4
scenario_03_opening_l2.json	baseline	4	3	3	2	3	4

Limitations

This review has two clear limitations. First, it is a small qualitative sample rather than a full annotation of all scenarios and all runs. Second, the author of the system carried out the review, which introduces an obvious risk of bias. For that reason, the results should be read as interpretive support for the broader evaluation, not as an independent final benchmark.

A Proposed User Study

A user study with students interested in consulting recruitment would help evaluate the system further. One option would be to collaborate with the University of St Andrews Consulting Club, since its members actively prepare for consulting case interviews and represent a relevant group of potential users.

The study would aim to understand whether the system feels like a realistic interview and whether the feedback it generates actually helps participants improve their case interview performance. It would focus on the user's perspective, complementing the technical evaluation of the system carried out earlier.

Participation would be voluntary and anonymous. Before taking part, participants would receive information about the purpose of the study, what data would be collected, and how the results would be used. They would give informed consent before starting, and no personally identifiable information would be collected.

Study Procedure

Participants would complete a simulated consulting case interview with the proposed system. To make the interaction feel closer to a real interview, the participant would not interact directly with the system's output. Instead, a person would act as interviewer, reading aloud the questions and prompts the AI interviewer agent generates and passing them on to the participant. This would preserve the conversational nature of a real case interview while still letting the system be tested in a realistic setting.

The proposed procedure would be the following.

1. Participants would receive a short introduction explaining the format of the case interview.

2. The interviewer would present the case and pass on the questions the AI system generates.

3. Participants would work through the case, explaining their reasoning, asking for additional information when needed, and giving a final recommendation.

4. After the conversation, the system would generate personalised feedback based on the interview transcript and how it evaluated the participant's answers.

5. Participants would fill in an anonymous questionnaire about their experience.

The whole session would take around 30–45 minutes.

Questionnaire Design

The questionnaire would first ask about the participant's previous experience with consulting interviews, then move on to questions about how well the system performed.

The background questions could include the following.

- What is your academic background?
- How familiar are you with consulting case interviews?
- Have you previously prepared for consulting interviews?
- Have you used AI tools for interview preparation before?

The main questionnaire would use a five-point Likert scale, ranging from 1 (Strongly disagree) to 5 (Strongly agree). Example statements are shown below.

- The interview experience felt similar to a real consulting case interview.
- The interviewer adapted the conversation according to my answers.
- The feedback accurately reflected my performance.
- The feedback identified specific areas where I could improve.
- The feedback provided practical suggestions that I could apply in future interviews.
- I believe repeated practice with this system could improve my case interview skills.
- I would consider using this system as part of my preparation for consulting interviews.

Participants would also have the chance to give qualitative feedback through open-ended questions, such as the following.

- What was the most useful aspect of the experience?
- What parts of the interview or feedback could be improved?
- Did any part of the feedback seem unclear or inaccurate?

The responses would show whether the system can deliver a realistic and valuable practice environment, and would also reveal limitations that automated evaluation alone might miss.

A Testing

This appendix summarises the code-level testing carried out during the project. It does not cover interview quality but checks that the main software components behave correctly and stay stable as the code changes.

A.1 Testing Scope

The automated tests focus on deterministic parts of the system, namely graph state handling, turn progression, retrieval plumbing, persistence, metric calculation, and HTTP endpoints. External model calls are isolated using synthetic fixtures, mocks, and temporary SQLite databases, which keeps the suite fast and failures easy to interpret.

A.2 Automated Tests

The main test files are `src/main/studio/tests/test_agentic_graph.py`, `src/main/studio/tests`
`src/main/studio/tests/test_api_connection.py`, and `src/main/web/tests/test_workbench.p`

A.2.1 Studio Logic

Three small fixtures, `make_runtime_case()`, `make_runtime_bundle()`, and `make_state()`, build a synthetic case with blocks such as `prompt_1`, `data_1`, `guide_1`, and `rec_1`, so the graph can be exercised without production data. Model calls are replaced with a local `mock_llm`, injected via patches such as `patch.object(agentic, "interviewer_llm", mock_llm)` or `patch.object(agentic, "judge_llm", mock_llm)`. On top of this, the studio tests cover the core conversation graph for both the agentic and baseline architectures. They check the construction of the initial state, the first interviewer turn, transcript visibility rules for the candidate, use of judge `focus_areas`, round- and turn-budget control, controlled fallback after invalid JSON output, and retrieval of case-guide context, alongside a full end-to-end run with mocked dependencies. `test_candidate_visible_`
`test_interviewer_retries_invalid_json_before_controlled_fallback`, and `test_judge_scou`
are representative examples. A smaller file also tests helper logic used in the RAG ablation analysis, such as stripping evaluation-only lines from transcripts before analysis.

A.2.2 Workbench and Persistence

`create_runs_table()` builds the `runs` and `agent_state_traces` tables in a temporary SQLite database, and `insert_sample_run()` inserts a minimal run identified as `run_001`, with a sample transcript containing lines such as `"Interviewer: Our profits are falling."` and `"Candidate: I would split revenue and costs."` The database path is redirected with `patch.object(dashboard_store, "RUNS_DB_PATH", self.db_path)`, and Flask's `test_client()` calls real routes such as `/runs/run_001?format=json`, `/api/runs/run_001/human-evaluation`, and `/runs/run_001/trace?format=json`. The tests check metric computation, run loading, trace reconstruction, cached golden-set loading, the human-evaluation save/read cycle, deletion flows, and API input validation, for example `test_load_run_traces_builds_timeline_payload` and `test_delete_batch_rej`

A.3 Integration and Smoke Tests

The project also includes connectivity smoke tests for the configured LLM endpoints, meant to run before launching the full system so invalid API keys or unreachable servers get caught early. The helper `_check_endpoint(base_url, api_key)` sends an `httpx.get()` request to each `/models` endpoint with `CONNECT_TIMEOUT = 5.0`, covering `lmstudio_llm_server`, `openai_llm_server`, `interviewer_llm_server`, `candidate_llm_serve` `judge_llm_server`, and `feedback_llm_server`. These checks only confirm reachability and basic authentication, not the quality of what each service generates.

A.4 Current Limitations

Most tests rely on mocks, so they say more about control flow and data handling than about the semantic quality of model outputs, and the smoke tests only confirm that external services respond, not that their generations stay stable or pedagogically sound. The separate evaluation methodology described in Chapter 4 complements code testing for that reason.

A User Manual

This chapter covers installing and running the prototype, environment setup, launching interview batches, and the local workbench used to inspect and annotate runs. Code correctness is Chapter A; interview-quality methodology is Chapter 4. This one is just operations.

Commands are run from `src/` unless noted otherwise.

A.1 Requirements and Installation

You need Python 3.10+ and an OpenRouter API key. All four active LLM roles go through OpenRouter.

Command	Effect
`make setup`	Alias for `make setup-local`. Creates `.venv/` if missing, installs `requirements.txt` and the LangGraph CLI (`langgraph-cli[inmem]`), writes `RUNNER := local` to `src/.make.runner.mk`.
`make setup-hpc`	Creates or reuses a conda environment `coach` (Python 3.11), installs the same dependencies there, writes `RUNNER := hpc` so later targets (`test`, `run-all`, `workbench`, ...) run inside it.

`make setup-local` switches back. Not sure which one is active? Check `src/.make.runner.mk`.

A repository-level `.env` is loaded automatically. API keys, model choices, and turn budgets all live there (Section A.2).

A.2 Configuring the LLM Roles

`agentic` uses four separate LLM roles. `baseline` fuses Interviewer, Judge and grader into one node. Model and temperature are set per role in `.env`.

Role	Model variable	Temperature variable	Used by
Interviewer	OPENROUTER_MODEL_INTERVIEWER	INTERVIEWER_TEMPERATURE	agentic
Candidate	OPENROUTER_MODEL_CANDIDATE	CANDIDATE_TEMPERATURE	both
Judge	OPENROUTER_MODEL_JUDGE	JUDGE_TEMPERATURE	agentic
Feedback	OPENROUTER_MODEL_FEEDBACK	FEEDBACK_TEMPERATURE	both
Baseline (fused)	OPENROUTER_MODEL_BASELINE	BASELINE_TEMPERATURE	baseline

Turn and round budgets work the same way. MAX_INTERVIEWER_TURNS_BEFORE_JUDGE caps interviewer turns before the agentic judge steps in. MAX_JUDGE_ROUNDS caps judge-checkpoint rounds. MAX_BASELINE_TURNS is the baseline equivalent. MAX_INTERVIEWER_JSON_RETRIES and MAX_BASELINE_JSON_RETRIES cap retries on invalid structured output before the fallback described in Chapter A kicks in.

LMSTUDIO_* variables are read by llm_server.py, but no graph role actually uses them. They only back the connectivity smoke test (Section A.3) and can stand in as a local endpoint during development.

A.3 Running Interview Batches

run_all_scenarios.py simulates interviews across synthetic scenarios with baseline and/or agentic, then writes a SQLite record and a batch export folder.

Flag	Default	Meaning
-graph	both	baseline, agentic, or both.
-limit	0 (all)	Max scenarios per graph.
-scenario	none	Run only this scenario; repeatable. Overrides -case.
-case	none	Run every scenario under this case id.
-repeat	4	Repetitions per scenario.
-seed	0	Seed for the initial-state builders.
-output-dir	main/artifacts/batch_runs/<timestamp>/	Custom output directory.
-label	none	Suffix on the batch folder name.

```
python3 run_all_scenarios.py --graph both --limit 3
python3 run_all_scenarios.py --graph agentic --case 01-energy-company --repeat 10
```

Scripted case ids are `01-energy-company`, `02-football-team`, and `03-agriculture-company`. There's a fourth asset, `04-worldcup-test`, sitting in `src/synthetic-dataset/`, but no scenario points to it as a `case_id`. It backs the `worldcup` golden-set fixtures instead (Section A.5).

Makefile shortcuts:

```
make run-all                                  # both graphs, all scenarios [N=][LABEL=]
make run-all-baseline                         # baseline only
make run-all-agentic                          # agentic only
make experiment CASE=<case_id> N=<repeat> [GRAPH=agentic|baseline|both] [LABEL=...]
```

`make experiment` requires `CASE`. `N` defaults to `4`, `GRAPH` to `agentic`.

Each batch adds one persisted run per scenario to `main/artifacts/runs.sqlite` and a snapshot under `main/artifacts/batch_runs/<timestamp>[_label]/`. Depending on `-graph`, that snapshot holds some combination of `baseline_results.jsonl/.csv`, `agentic_results.jsonl/.csv`, `combined_results.jsonl/.csv` and `summary.json`. Records carry the graph name, scenario reference, full transcript, final feedback, case-performance and dialogue-quality scores with rationales, and error status when a run fails.

A.4 The Workbench

A local Flask app for browsing runs and batches, and for adding human evaluation.

```
make workbench
```

That's `cd main/web && WORKBENCH_PORT=5020 python3 app.py`, served at `http://localhost:5020`. Change the port with `WORKBENCH_PORT=5050 make workbench`. Nav bar: Runs, Experiment, RAG Evaluation, Agents.

A.4.1 Runs and run detail

`/` lists the 100 most recent runs. `/runs/<run_id>` shows transcript, reference scores, model scores, derived metrics (`?format=json` for the raw payload). `/runs/<run_id>/trace` reconstructs the agent-state timeline, same JSON option. Deleting posts to `/runs/<run_id>/delete`.

A.4.2 Compare

`/compare?run_a=<id>&run_b=<id>` shows two runs from the same list, side by side.

A.4.3 Human evaluation

The evaluation form on the run-detail page posts to `/runs/<run_id>/human-evaluation`. It collects evaluator name, free-text notes, an overall score, and a score plus rationale for every dimension already scored on that run, both case performance (`case_opening`, `case_structure`, `final_recommendation`, ...) and dialogue quality (`groundedness`, `clarity_and_concision`, `multi_turn_coherence`, ...). Same data as JSON through `GET/POST /api/runs/<run_id>/human-evaluation`. Model-vs-reference scores at `GET /api/runs/<id>/scores`.

A.4.4 Experiment

`/experiment` aggregates a whole batch folder. Pick one with `?batch=<dir_name>` (defaults to the most recent), narrow with repeated `?ref=<n>`. Export as zip (`/experiment/<dir_name>/download`), delete (`POST /experiment/<dir_name>/delete`), or drill into a single scenario at `/experiment/<dir_name>/<slug>`.

A.4.5 RAG Evaluation

`/rag-evaluation` runs the retrieval layer directly. `?k=<n>` sets the number of retrieved passages (default `DEFAULT_TOP_K`), `?refresh=1` forces a fresh retrieval instead of the cached one. The same page shows a batch's RAG-ablation comparison (Section A.5) via `?batch=<dir_name>` and `?ablation_refresh=1`.

A.4.6 Agents

`/agents` links to `/agents/judge` and `/agents/interviewer`. Both take `?golden_set=<name>` and `?refresh=1`.

A.5 Node-Level Evaluation Suites

These suites test individual nodes, judge, interviewer, and baseline, against hand-labelled golden-set CSVs, with no full batch required. Results land as JSON next to the CSVs and show up on the Agents pages too.

Command	What it does
`make judge-eval [GOLDEN_SET=worldcup]`	Runs a judge golden-set CSV against the real judge LLM, scores `enough_evidence` accuracy.
`make interviewer-eval [INTERVIEWER_GOLDEN_SET=...]`	Runs an interviewer golden-set CSV (`evidence_handling` by default, or `socratic_function`, `guardrail`, `turn_control`).
`make baseline-eval [BASELINE_GOLDEN_SET=...]`	Same fixtures plus `worldcup`, run against the baseline fused node, for comparison with the agentic interviewer and judge.
`make rag-ablation BATCH=<dir_name>`	Replays a batch's stored transcripts through the judge eval nodes with retrieval disabled, compares `eval_case`/`eval_dialog` against the original with-RAG scores. `BATCH` has to already exist under `main/artifacts/batch_runs/`.
`make golden-set-sync-check`	No-LLM check that every baseline golden-set CSV still covers the same `conversation_id`s as the judge/interviewer CSV it mirrors.
`make node-eval-all [LIMIT=<n>]`	Full sweep, run sequentially so results stay attributable and OpenRouter rate limits don't get hit in parallel. It runs the sync check, judge-eval (`worldcup`), interviewer-eval ×4, and baseline-eval ×5, in that order.

All targets take `LIMIT=<n>` to cap how many golden-set rows get evaluated.

A.6 LangGraph Dev Server

`make langgraph`

`cd main/studio && langgraph dev -no-browser`. Opens the graph definitions in LangGraph Studio.

A.7 Automated Tests

`make test` runs the studio and workbench unit-test suites (Chapter A). `make test-api-connection` runs just the connectivity smoke test against the configured LLM endpoints.

Bibliography

[1] K. Anders Ericsson, Ralf T. Krampe and Clemens Tesch-Römer. 'The Role of Deliberate Practice in the Acquisition of Expert Performance'. In: *Psychological Review* 100.3 (1993), pp. 363–406. DOI: 10.1037/0033-295X.100.3.363.

[2] Jenny Rae Le Roux and Kevin Gao. *The Consulting Interview Bible*. https://managementconsulted.com/app/uploads/2014/10/The-Consulting-Interview-Bible-2015-0928-2.pdf. Accessed: 2026-07-05. 2015.

[3] Mingzhe Li et al. *EZInterviewer: To improve job interview performance with mock interview generator*. 2023. arXiv: 2301.00972.

[4] University of Oxford Careers Service. *Consulting Case Study Interviews*. 2026. URL: https://www.careers.ox.ac.uk/consulting-case-study-interviews (visited on 05/06/2026).

[5] Boston Consulting Group. *Case Interview Preparation*. 2026. URL: https://careers.bcg.com/global/en/case-interview-preparation (visited on 05/06/2026).

[6] Michael Wooldridge and Nicholas R. Jennings. 'Intelligent Agents: Theory and Practice'. In: *The Knowledge Engineering Review* 10.2 (1995), pp. 115–152. DOI: 10.1017/S0269888900008122.

[7] Joon Sung Park et al. 'Generative agents: Interactive simulacra of human behavior'. In: *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*. Association for Computing Machinery, 2023. DOI: 10.1145/3586183.3606763.

[8] Guohao Li et al. 'CAMEL: Communicative agents for "mind" exploration of large language model society'. In: *Advances in Neural Information Processing Systems*. 2023.

[9] Aske Plaat et al. 'Agentic Large Language Models, a survey'. In: *arXiv preprint arXiv:2503.23037* (2025). URL: https://arxiv.org/abs/2503.23037.

[10] Truong Thanh Hung Nguyen et al. *SimInterview: Transforming business education through large language model-based simulated multilingual interview training system*. 2025. arXiv: 2508.11873.

[11] Eunsu Kim et al. *LLM-as-an-interviewer: Beyond static testing through dynamic LLM evaluation*. 2025.

[12] Lucile Favero et al. *Enhancing critical thinking in education by means of a Socratic chatbot*. 2024. arXiv: 2409.05511.

[13] Jiawei Gu et al. *A Survey on LLM-as-a-Judge*. 2024. arXiv: 2411.15594.

[14] Yunfan Gao et al. 'Retrieval-Augmented Generation for Large Language Models: A Survey'. In: *arXiv preprint arXiv:2312.10997* (2023).

[15] *AGSM Case Book*. https://careerinconsulting.com/wp-content/uploads/2019/12/2.-AGSM-case-book-2002.pdf. Accessed: 2026-07-27. 2002.

[16] Duke Fuqua School of Business. *Fuqua Case Book*. https://careerinconsulting.com/wp-content/uploads/2019/12/11.-Fuqua-Case-book-2017.pdf. Accessed: 2026-07-27. 2017.

[17] Harvard Business School. *Harvard Case Book*. https://careerinconsulting.com/wp-content/uploads/2019/12/4.-Harvard-case-book.pdf. Accessed: 2026-07-27.

[18] Career in Consulting. *Case Interview Examples*. https://careerinconsulting.com/case-interview-examples/. Accessed: 2026-07-27. 2026.

[19] Patty Goedl. *Principles of Managerial Accounting*. Licensed under CC BY 4.0. Cincinnati, OH: University of Cincinnati Pressbooks, 2020. URL: https://ucincinnatipress.pressbooks.pub/principlesaccounting/.

[20] Xiachong Feng, Longxu Dou and Lingpeng Kong. *Reasoning Does Not Necessarily Improve Role-Playing Ability*. Accepted at Findings of the Association for Computational Linguistics: ACL 2025. 2025. arXiv: 2502.16940 [cs.CL]. URL: https://arxiv.org/abs/2502.16940.

[21] Langfuse. *LLM Observability and Application Tracing*. 2026. URL: https://langfuse.com/docs/observability/overview (visited on 10/08/2026).

[22] Helia Hashemi et al. *LLM-RUBRIC: A Multidimensional, Calibrated Approach to Automated Evaluation of Natural Language Texts*. Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 13806–13834. 2024.

[23] Jason Wei et al. 'Chain-of-Thought Prompting Elicits Reasoning in Large Language Models'. In: *arXiv preprint arXiv:2201.11903* (2022).

[24] Jacob Cohen. 'Weighted Kappa: Nominal Scale Agreement with Provision for Scaled Disagreement or Partial Credit'. In: *Psychological Bulletin* 70.4 (1968), pp. 213–220. DOI: 10.1037/h0026256.

[25] Marina Sokolova and Guy Lapalme. 'A systematic analysis of performance measures for classification tasks'. In: *Information Processing & Management* 45.4 (2009), pp. 427–437. DOI: 10.1016/j.ipm.2009.03.002.

[26] Gemma Team. *Gemma 3 Technical Report*. 2025. arXiv: 2503.19786 [cs.CL]. URL: https://arxiv.org/abs/2503.19786.

[27] DAIR.AI. *LLM Settings*. Prompt Engineering Guide. Accessed 2026. 2024. URL: https://www.promptingguide.ai/introduction/settings.

School of Computer Science
Student Project Ethics Form

Title of project

Designing an AI-Based Simulator for Consulting Case Interviews

Name of student

Paloma Pérez De Madrid

Name of supervisor

Dr Ognjen Arandelovic

To be completed by student

I confirm that:

(select one of the following options)

[1] ☐ My **intended project** raises no ethical issues: I have agreed with **all** of the statements on the following page, in relation to my intended project.

[2] ☐ My **intended project** is covered by the Computer Science generic artifact evaluation ethical application (approval code CS15727).

[3] ☐ My **fall-back project** raises no ethical issues: I have agreed with **all** of the statements on the following page, in relation to my fall-back project. I will submit a full ethics application for my intended project.

[4] ☒ My **fall-back project** raises no ethical issues: I have agreed with **all** of the statements on the following page, in relation to my fall-back project. My supervisor will submit an ethics amendment application for my intended project.

[5] ☐ My **fall-back project** is covered by approval CS15727. I will submit a full ethics application for my intended project.

[6] ☐ My **fall-back project** is covered by approval CS15727. My supervisor will submit an ethics amendment application for my intended project.

Signature of student



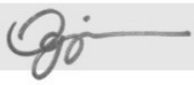
Date

06 of April of 2026

To be completed by supervisor

I confirm that I approve the student's statement above.

Signature of supervisor



Date

6 APR 2026